Guida a Go

Dal programma «Hello, World» alla concorrenza

Guida tecnica e didattica al linguaggio Go
per studenti di Ingegneria Informatica

Il contenuto è ricavato dalle slide *Introduction to Go*
del corso di *Sistemi Distribuiti e Cloud Computing*, A.A. 2025/26,
Laurea Magistrale in Ingegneria Informatica, Università di Roma Tor Vergata
(autrice delle slide: Valeria Cardellini).

Gli esempi di codice di questa guida si trovano nella cartella `examples/`
e sono stati compilati ed eseguiti con Go 1.27.3.

Indice

Introduzione e obiettivi **1**

1 Il linguaggio Go in sintesi **3**

1.1 Obiettivi di progetto . . . 3

1.2 Che cosa Go eredita da C . . . 3

1.3 Le novità rispetto a C . . . 4

1.4 Go per i sistemi distribuiti e il cloud . . . 4

1.5 Ambiente di sviluppo . . . 5

2 Struttura del programma, package e toolchain **6**

2.1 Il primo programma . . . 6

2.2 Package e visibilità . . . 7

2.3 Importare package . . . 7

2.4 La toolchain di Go . . . 8

2.5 Moduli . . . 9

2.6 Regole del compilatore e strumenti di qualità . . . 9

3 Variabili e tipi **11**

3.1 Dichiarare variabili . . . 11

3.2 Tipi di base, conversioni e inferenza . . . 13

4 Funzioni **15**

4.1 Dichiarazione e parametri . . . 15

4.2 Risultati multipli . . . 16

4.3 Valori di ritorno con nome . . . 17

4.4 Funzioni variadiche . . . 18

4.5 Funzioni anonime e closure . . . 19

5 Controllo di flusso **24**

5.1 Il ciclo `for` . . . 24

5.2 Istruzioni condizionali: `if` . . . 26

5.3 Istruzioni condizionali: `switch` . . . 27

5.4 L'istruzione `defer` . . . 28

6 Puntatori e strutture dati **31**

6.1 Puntatori . . . 31
6.2 Strutture dati aggregate . . . 33
6.3 Slice . . . 35
6.4 Map . . . 40
6.5 L'istruzione `range` . . . 42

7 Metodi, interfacce ed errori **45**

7.1 Metodi . . . 45
7.2 Interfacce . . . 47
7.3 Gestione degli errori . . . 50

8 Concorrenza **53**

8.1 La concorrenza in Go . . . 53
8.2 Goroutine . . . 53
8.3 Channel . . . 58
8.4 Channel con e senza buffer . . . 61
8.5 Chiudere un channel e iterare con `range` . . . 62
8.6 `select` . . . 66
8.7 Timer . . . 69
8.8 Mutua esclusione . . . 71
8.9 Sincronizzazione con `WaitGroup` . . . 76
8.10 Priorità e scheduling . . . 78
8.11 Deadlock . . . 79
8.12 Riepilogo degli errori comuni nella concorrenza . . . 81

9 Test e benchmark **82**

9.1 Il package `testing` . . . 82
9.2 Benchmark . . . 84
9.3 Argomenti citati ma non sviluppati . . . 86

Conclusione e riepilogo **87**

A Mappa tra slide, guida ed esempi **89**

B Note della verifica incrociata **91**

B.1 Metodo . . . 91
B.2 Errori e imprecisioni trovati nelle slide . . . 91

B.3 Contenuto delle slide deliberatamente omesso 92
B.4 Verifica degli esempi 92

Introduzione e obiettivi

A chi si rivolge e che cosa contiene

Questa guida trasforma le slide del corso in un testo organico per lo studio individuale. Presuppone che il lettore conosca già la programmazione e abbia familiarità con C, Python e Java: i concetti generali (variabili, cicli, funzioni, ricorsione, …) non vengono rispiegati. L'attenzione è invece sulle scelte specifiche di Go e su come si collocano rispetto ai linguaggi già noti.

Gli argomenti sono esattamente quelli delle slide, in una successione leggermente riordinata per rendere l'apprendimento più naturale (per esempio i moduli sono trattati insieme alla toolchain, e la gestione degli errori insieme alle interfacce). Sono stati omessi, perché non contribuiscono all'apprendimento tecnico, i cenni storici, le classifiche di popolarità dei linguaggi e i riferimenti bibliografici o web. Quando una slide accenna soltanto a un argomento (per esempio RPC o il package `net/http`), la guida lo segnala ma non lo sviluppa.

Come è organizzata la guida

Cap.	Argomento	Cartella degli esempi
1	Il linguaggio Go in sintesi	—
2	Struttura del programma, package, toolchain, moduli	`01_basics`
3	Variabili e tipi	`01_basics`
4	Funzioni, closure, funzioni variadiche	`02_functions`
5	Controllo di flusso e `defer`	`03_control_flow`
6	Puntatori e strutture dati	`04_data_structures`
7	Metodi, interfacce, errori	`05_methods_interfaces`
8	Concorrenza	`06_concurrency`
9	Test e benchmark	`07_testing_and_tools`

Le due appendici raccolgono la mappa tra slide, sezioni ed esempi (Appendice A) e le note della verifica incrociata, comprese le correzioni a errori trovati nelle slide (Appendice B).

Come usare gli esempi

Ogni listato della guida riporta il nome del file corrispondente nella cartella `examples/`. Il testo dei listati è incluso *automaticamente* dai file `.go` reali, quindi guida ed esempi non possono divergere. Anche gli output mostrati sono stati ottenuti eseguendo i programmi; quando l'output non è deterministico, il testo lo dice esplicitamente.

Ogni file è un programma indipendente (`package main`) da eseguire dalla cartella `examples/`, che contiene un file `go.mod` condiviso:

```
cd examples
go run 01_basics/helloworld.go
go run 03_control_flow/echo.go Hello Go World
```

Poiché ogni cartella contiene più programmi con un proprio `main`, non si usa `go build ./...` sull'intera cartella (il compilatore segnalerebbe `main redeclared`): si esegue un file alla volta. L'unica eccezione è la cartella `07_testing_and_tools`, che è un package ordinario testato con `go test` (Capitolo 9). Serve Go 1.22 o successivo (si veda la nota sulle variabili di ciclo nel Capitolo 8).

Convenzioni grafiche

Definizione — esempio

Un box blu introduce una definizione precisa di un concetto.

Osservazione

Un box verde raccoglie un'osservazione importante da ricordare.

Errore comune

Un box rosso descrive un errore frequente, spiegando *perché* avviene e come evitarlo.

Confronto — C, Python, Java

Un box viola mette a confronto il costrutto Go con quelli dei linguaggi noti.

I blocchi con sfondo giallo e barra arancione riportano l'output reale dei programmi.

Capitolo 1

Il linguaggio Go in sintesi

Prima di scrivere codice conviene capire *che tipo di linguaggio* è Go e quali problemi i suoi progettisti hanno voluto risolvere. Molte scelte che a prima vista sembrano restrizioni (niente eccezioni, niente classi, niente aritmetica dei puntatori, regole di formattazione rigide) sono conseguenze dirette di questi obiettivi.

1.1 Obiettivi di progetto

Go è definito come un linguaggio open source che rende facile costruire sistemi *semplici*, *sicuri* e *scalabili*. In termini più tecnici, gli obiettivi dichiarati sono tre:

1. essere **espressivo ed efficiente** sia in compilazione sia in esecuzione;
2. favorire la scrittura di programmi **affidabili e robusti**;
3. offrire un linguaggio **staticamente tipato e compilato rapidamente**, ma che *si usi* come un linguaggio a tipizzazione dinamica e interpretato.

L'ultimo punto è il più interessante. Un programmatore Python scrive poche righe e ottiene subito un risultato, ma scopre molti errori solo a runtime; un programmatore C ottiene prestazioni elevate ma deve annotare tutto e gestire manualmente la memoria. Go cerca un compromesso: ogni variabile ha un tipo statico verificato dal compilatore, ma l'*inferenza di tipo* (Capitolo 3) evita di ripeterlo continuamente, e la compilazione è abbastanza veloce da rendere il ciclo modifica–esecuzione simile a quello di un linguaggio interpretato (`go run`, Sezione 2.4).

1.2 Che cosa Go eredita da C

Go è spesso descritto come un linguaggio «C-like» o «il C per il ventunesimo secolo». Dal C eredita:

- la **sintassi delle espressioni**;
- le **istruzioni di controllo del flusso** (`for`, `if`, `switch`);
- i **tipi di dato di base**;
- il **passaggio dei parametri per valore** (*call-by-value*): ogni argomento è copiato nel parametro formale;
- i **puntatori**;
- l'**efficienza a runtime**;
- la **tipizzazione statica**.

Per la concorrenza, invece, l'antenato dichiarato è il modello **CSP** (*Communicating Sequential Processes*, formulato da Tony Hoare): processi sequenziali indipendenti che cooperano scambiandosi messaggi anziché condividendo memoria. Questo modello è implementato da goroutine e channel (Capitolo 8).

1.3 Le novità rispetto a C

Le slide raggruppano le funzionalità moderne di Go in quattro aree.

Concorrenza. Strumenti nuovi ed efficienti per la programmazione concorrente, parte del linguaggio e non di una libreria esterna.

Astrazione dei dati e programmazione a oggetti. Un approccio flessibile basato sulla *composizione* piuttosto che sull'ereditarietà. Go non ha classi: il comportamento si associa a un tipo con i *metodi* e si astrae con le *interfacce* (Capitolo 7).

Gestione della memoria. Gestione automatica con *garbage collection*: il programmatore non libera esplicitamente la memoria.

Leggibilità e usabilità. Sintassi pulita e concisa, pensata per codice manutenibile e leggibile.

Confronto — gestione della memoria

In **C** la memoria dinamica si alloca con `malloc` e si libera con `free`; errori come il *use-after-free* o i *memory leak* sono a carico del programmatore. **Java** e **Python** hanno anch'essi un garbage collector (Python in CPython combina conteggio dei riferimenti e rilevazione dei cicli). **Go** si colloca dalla parte di Java e Python: il runtime libera automaticamente gli oggetti non più raggiungibili, eliminando in particolare i problemi di use-after-free. A differenza di Java e Python, però, Go espone i puntatori (senza aritmetica, Sezione 6.1) e produce codice nativo compilato, senza macchina virtuale né interprete.

1.4 Go per i sistemi distribuiti e il cloud

Le slide motivano Go nel contesto dei sistemi distribuiti: il linguaggio permette di concentrarsi sui problemi del distribuito grazie a

- un buon supporto alla **concorrenza**;
- un buon supporto alle **RPC** (*Remote Procedure Call*);
- il **garbage collector**, che evita i problemi di *use-after-free*;
- la **sicurezza dei tipi** (*type safety*);
- la facilità di apprendimento e manutenzione.

Go è inoltre sempre più usato per le applicazioni *cloud-native*. A titolo di esempio, le slide citano la libreria *Go Cloud*, che fornisce astrazioni sui servizi dei diversi fornitori cloud: lo stesso codice può leggere un oggetto da un *blob storage* indipendentemente dal provider sottostante, cambiando solo l'URL del bucket. Il frammento seguente riproduce quello della slide.

Listato 1.1: Frammento della slide sul Go Cloud (non eseguibile in isolamento)

```
1 ctx := context.Background()
```

```
bucket, err := blob.OpenBucket(ctx, "s3://my-bucket")
if err != nil {
    return err
}
defer bucket.Close()
blobReader, err := bucket.NewReader(ctx, "my-blob", nil)
if err != nil {
    return err
}
```

Perché questo frammento non ha un file in `examples/`

Il codice richiede la libreria esterna *Go Cloud*, credenziali del provider e un bucket esistente: non è eseguibile né verificabile in modo autonomo, e usarlo come esempio introdurrebbe argomenti esterni al programma. Il frammento è utile soprattutto per riconoscere i pattern già visti: la coppia risultato/errore (Sezione 7.3), il controllo `err != nil` e l'istruzione `defer` per la pulizia (Sezione 5.4).

1.5 Ambiente di sviluppo

Go si può scrivere con un qualsiasi editor di testo; gli ambienti indicati dalle slide sono **GoLand** (IDE di JetBrains), **Visual Studio Code** e, per chi usa vim o neovim, il plugin **vim-go**.

Dietro le quinte tutti questi strumenti si appoggiano a **gopls**, il *language server* ufficiale di Go. Un language server implementa il *Language Server Protocol* (LSP), uno standard con cui un editor ottiene da un processo separato le funzioni avanzate di supporto al linguaggio: completamento del codice, documentazione al passaggio del mouse, ricerca dei simboli, diagnostica ed errori. Il vantaggio architetturale è che l'intelligenza sul linguaggio è scritta una sola volta (in `gopls`) e riutilizzata da qualunque editor che parli LSP.

Capitolo 2

Struttura del programma, package e toolchain

2.1 Il primo programma

2.1.1 Esempio pratico

Listato 2.1: `01_basics/helloworld.go`

```go
// helloworld.go - Primo programma Go (slide 8-9).
//
// Ogni file inizia con la dichiarazione del package. Il package "main" è
// speciale: dice al toolchain di produrre un eseguibile, il cui punto di
// ingresso è la funzione main().
package main

// "fmt" fornisce l'I/O formattato (Println, Printf, ...).
import "fmt"

func main() {
    // Le stringhe Go sono UTF-8 nativamente: nessuna configurazione
    // particolare serve per stampare caratteri non ASCII.
    fmt.Println("Hello, 世界")
}
```

```
Hello, 世界
```

2.1.2 Analisi del codice

- **Nessun punto e virgola** a fine istruzione o dichiarazione. In realtà il compilatore li inserisce automaticamente a fine riga in certi casi (si veda l'errore comune più avanti); il programmatore non li scrive.
- **Supporto Unicode nativo**: il sorgente è UTF-8 e le stringhe contengono testo Unicode, come mostra la stringa `"Hello, 世界"`.
- **Struttura a package**: ogni programma Go è composto da *package* (concetto analogo alle librerie C o ai moduli Python). Un package è costituito da uno o più file `.go` nella stessa directory.
- **Struttura di un file sorgente**: inizia con la *dichiarazione del package* (a quale package appartiene il file), seguita dagli `import` degli altri package.
- **Punto di ingresso**: l'esecuzione parte dal package `main` e dalla sua funzione `main()`.

- Il package `fmt` fornisce le funzioni per l'input e l'output formattato (`Println`, `Printf`, …).

Errore comune: la graffa `{` a capo

Poiché i punti e virgola sono inseriti dal compilatore a fine riga, la graffa aperta deve stare sulla stessa riga della parola chiave. In particolare in un `if` / `else` l'`else` deve comparire subito dopo la `}` che chiude il blocco precedente. Il codice

```
if x > 0 {
    fmt.Println(1)
}
else {
    fmt.Println(2)
}
```

non compila: dopo la `}` della riga 3 il compilatore inserisce un punto e virgola, l'istruzione `if` risulta conclusa, e la riga successiva inizia con un `else` inatteso. Il messaggio reale è: `syntax error: unexpected keyword else, expected }`.

2.2 Package e visibilità

Un programma Go è composto da package, e l'esecuzione parte dal package `main`. Un package può contenere dichiarazioni di tipi, funzioni, variabili e costanti, e può essere molto piccolo o molto grande.

Definizione — regola di visibilità

In Go la **visibilità** di un nome fuori dal package che lo dichiara è determinata dalla *maiuscola iniziale*. Un nome che inizia con una lettera **maiuscola** è *esportato* (pubblico); uno che inizia con una lettera **minuscola** non è esportato (privato al package).

Così `Foo` è esportato e `foo` no. Un esempio concreto: `math.Pi` è visibile da altri package, mentre `math.pi` non lo è, e usarlo produce un errore di compilazione. Le slide riportano il messaggio `./prog.go:9:19: undefined: math.pi`; il compilatore usato per verificare questa guida (Go 1.27.3) lo completa con un suggerimento:

```
undefined: math.pi (but have Pi)
```

Confronto — C, Python, Java

In **Java** la visibilità si controlla con le parole chiave `public`/`private`/`protected`; in **C** con `static` (a livello di file) e con ciò che si dichiara negli header; in **Python** vale solo la convenzione del prefisso `_`. In **Go** la regola è *imposta dal compilatore* ed è codificata nel nome stesso: guardando un identificatore si sa subito se fa parte dell'interfaccia pubblica di un package.

2.3 Importare package

L'istruzione `import` include altri package. Gli import si possono raggruppare in una forma «fattorizzata» tra parentesi, più leggibile della ripetizione di tante istruzioni `import`.

Listato 2.2: `01_basics/imports.go`

```go
// imports.go - Import "fattorizzato" tra parentesi (slide 12) e visibilità
// dei nomi (slide 11).
package main

import (
    "fmt"
    "math"
)

func main() {
    // math.Sqrt restituisce un float64; %g sceglie la forma compatta
    // più breve tra %e (esponenziale) e %f (decimale).
    fmt.Printf("Now you have %g problems.\n", math.Sqrt(7))

    // Un nome esportato (iniziale maiuscola) è accessibile dall'esterno:
    // math.Pi è visibile, mentre math.pi non esiste e l'uso di math.pi
    // produce l'errore di compilazione "undefined: math.pi".
    fmt.Println("math.Pi =", math.Pi)
}
```

```
Now you have 2.6457513110645907 problems.
math.Pi = 3.141592653589793
```

Il formato `%g` stampa un numero in virgola mobile nella forma compatta, scegliendo automaticamente la più corta tra `%e` (esponenziale) e `%f` (decimale). Si noti inoltre che l'uso di `math.Sqrt(7)` richiede il prefisso del package (`math.`): i nomi importati non vengono mai «riversati» nello spazio dei nomi locale.

2.4 La toolchain di Go

Go è un linguaggio **compilato**. Il comando `go` è la *toolchain* con cui si scaricano, compilano e installano package e comandi. Un punto distintivo è la **configurazione zero**: non servono script di build esterni né makefile; il compilatore ricava le dipendenze dagli stessi file sorgente.

`go run` compila ed esegue direttamente il programma.

`go build` produce un eseguibile binario.

`go install` compila e installa package o comandi.

```
$ go run helloworld.go
Hello, 世界
$ go build helloworld.go
$ ls helloworld*
helloworld      helloworld.go
$ ./helloworld
Hello, 世界
```

Su Windows l'eseguibile prodotto da `go build` si chiama `helloworld.exe`. Il comando `go run` compila in una directory temporanea ed esegue il risultato: è comodo in fase di sviluppo, ma il binario non viene conservato.

Confronto — Java, Python, C

Rispetto a **Java**, il binario di Go è già codice macchina nativo: non richiede una JVM per essere eseguito. Rispetto a **C**, non servono `Makefile` o `CMake`: le dipendenze sono dichiarate dagli `import`. Rispetto a **Python**, la compilazione è un passo esplicito (`go build`) ma `go run` lo nasconde offrendo un ciclo modifica–esecuzione rapido.

2.5 Moduli

Definizione — modulo

Un **modulo** è un insieme di package Go correlati, memorizzati in una struttura di directory con un file `go.mod` nella radice.

Il file `go.mod` definisce:

- il **percorso del modulo** (*module path*), usato anche come percorso di import per la directory radice;
- la **versione minima di Go** richiesta dal modulo;
- le **dipendenze**: gli altri moduli necessari per una compilazione corretta, con la relativa versione minima.

Un esempio, come nelle slide:

Listato 2.3: Esempio di `go.mod` (dalla slide)

```
module example.com/mymodule

go 1.17

require github.com/gorilla/mux v1.8.0
```

I comandi principali sono:

```
$ go mod init <nome_modulo>   # crea il file go.mod
$ go mod tidy                 # aggiunge i requisiti mancanti e rimuove quelli
    inutilizzati
```

Nella cartella `examples/` di questa guida il file `go.mod` è minimo, perché gli esempi usano solo la libreria standard:

Listato 2.4: `examples/go.mod`

```
module goguide

go 1.22
```

2.6 Regole del compilatore e strumenti di qualità

Go è «pignolo»: alcune pratiche che in altri linguaggi producono al massimo un *warning* qui sono **errori di compilazione**. Il motivo è tenere il codice pulito per costruzione e rendere la lettura di sorgenti altrui più prevedibile.

- Una **variabile dichiarata e non usata** è un errore.
- Un **import non usato** è un errore.
- In un `if`/`else` la graffa `{` deve stare alla fine della stessa riga (`} else {`, `} else if … {`); si veda l'errore comune sopra.

I messaggi reali del compilatore sono:

```
declared and not used: x
"os" imported and not used
```

Per le variabili di cui non interessa il valore (per esempio l'indice di un ciclo quando serve solo l'elemento) si usa il **blank identifier** `_`, che scarta il valore e soddisfa il compilatore (Sezione 6.5).

Gli strumenti da riga di comando consigliati dalle slide sono:

`gofmt` formatta il codice Go secondo lo stile standard: `gofmt -w yourcode.go` riscrive il file formattato. Non esistono «guerre di stile»: il formato è uno solo.

`goimports` aggiunge e rimuove automaticamente gli import (utile proprio per evitare l'errore di import inutilizzato).

`godoc` permette di consultare la documentazione dei package.

Perché `gofmt` è importante

Tutti gli esempi di questa guida sono stati formattati con `gofmt`. In Go l'indentazione con tabulazioni, la posizione delle graffe e lo spazio attorno agli operatori non sono questioni di gusto personale: è la forma canonica che gli editor applicano al salvataggio.

Capitolo 3

Variabili e tipi

3.1 Dichiarare variabili

3.1.1 Introduzione

Go offre due modi per dichiarare una variabile: l'istruzione `var` e la *dichiarazione breve* `:=`. La prima si può usare sia a livello di package sia dentro le funzioni; la seconda solo dentro le funzioni.

3.1.2 Sintassi

- `var` dichiara una o più variabili. Il **tipo viene per ultimo**, dopo il nome: `var i int`.
- È ammesso un **inizializzatore** per ciascuna variabile; se è presente, il tipo si può omettere e viene dedotto: `var a, b = 1, "due"`.
- Una variabile dichiarata senza valore iniziale riceve il suo **valore zero**.
- La **dichiarazione breve** `count := 10` dichiara e inizializza deducendo il tipo; è consentita solo all'interno delle funzioni.

Definizione — valore zero

Ogni tipo di Go ha un **valore zero**, assegnato automaticamente a una variabile non inizializzata esplicitamente: `0` per i numeri, `false` per i booleani, la stringa vuota `""` per le stringhe e `nil` per puntatori e altri tipi riferimento.

3.1.3 Esempio pratico

Listato 3.1: `01_basics/variables.go`

```go
// variables.go - Dichiarazione di variabili e valori zero (slide 17).
package main

import "fmt"

// Variabili a livello di package: qui ":=" NON è ammesso, serve "var".
// Il tipo segue il nome; tutte e tre ricevono il valore zero (false).
var c, python, java bool

func main() {
    // Variabile locale senza inizializzatore: valore zero (0 per gli int).
    var i int
    fmt.Println(i, c, python, java)

    // Con inizializzatori (uno per variabile) il tipo può essere omesso
```

```
16      // e viene dedotto dal valore a destra.
17      var a, b = 1, "due"
18      fmt.Println(a, b)
19
20      // Dichiarazione breve ":=" - solo dentro le funzioni.
21      count := 10
22      fmt.Println(count)
23
24      // Valori zero dei tipi base: nessuna variabile Go è "non inizializzata".
25      var (
26          n int
27          f float64
28          s string
29          t bool
30          p *int
31      )
32      fmt.Printf("int=%d float64=%g string=%q bool=%t pointer=%v\n", n, f, s, t, p)
33  }
```

```
0 false false false
1 due
10
int=0 float64=0 string="" bool=false pointer=<nil>
```

3.1.4 Analisi del codice

Le variabili `c`, `python` e `java` sono dichiarate a livello di package con `var` (qui `:=` non sarebbe permesso). Un'unica dichiarazione, `var c, python, java bool`, assegna il tipo `bool` a tutte e tre; poiché nessuna ha un inizializzatore, valgono `false`. Dentro `main`, `var i int` vale `0`. L'ultima parte stampa i valori zero di tipi diversi (puntatore `nil` compreso), dimostrando che *in Go non esistono variabili non inizializzate.*

Confronto — C, Python, Java

In **C** una variabile locale non inizializzata ha un valore *indeterminato*: leggerla è un comportamento non definito e una classica fonte di bug. In **Java** i campi hanno un valore di default ma le variabili locali devono essere assegnate prima dell'uso (verificato dal compilatore). In **Python** non si dichiara nulla: il nome nasce con la prima assegnazione. In **Go** il valore zero è definito per tutti i tipi, quindi una variabile è sempre in uno stato valido. Per l'inferenza, `:=` ricorda `var` di Java (10+) per le variabili locali, ma in Go il tipo resta statico e fisso.

Errori comuni con `:=` e `var`

1. **Usare `:=` fuori da una funzione.** A livello di package ogni istruzione deve iniziare con una parola chiave di dichiarazione (`var`, `func`, …). Scrivere `x := 3` fuori da una funzione dà `syntax error: non-declaration statement outside function body`.
2. **Ridichiarare con `:=`.** `a := 1` seguito da `a := 2` nello stesso blocco dà `no new variables on left side of :=`: `:=` richiede almeno una variabile *nuova*; per riassegnare si usa `=`.

3. **Variabili inutilizzate:** sono errori, non warning (Sezione 2.6).

3.2 Tipi di base, conversioni e inferenza

I tipi di base comprendono `bool`, `string`, `int`, `uint`, `float32`, `float64` e altri.

Definizione — conversione esplicita

Go richiede una **conversione esplicita** tra tipi diversi, anche numerici: `var f float64 = float64(i)`. Non esiste alcuna promozione implicita (a differenza di C).

L'**inferenza di tipo** permette al compilatore di dedurre il tipo di una variabile dal valore a destra: se `var i int` è già dichiarata, allora in `j := i` la variabile `j` è di tipo `int`.

Listato 3.2: `01_basics/types.go`

```go
// types.go - Tipi base, conversioni esplicite e inferenza (slide 18).
package main

import "fmt"

func main() {
    // Tipi base.
    var (
        b  bool    = true
        s  string  = "go"
        i  int     = 42
        u  uint    = 7
        f2 float32 = 1.5
        f6 float64 = 2.25
    )
    fmt.Println(b, s, i, u, f2, f6)

    // Conversione esplicita: obbligatoria tra tipi diversi (a differenza
    // di C, dove int -> double avviene implicitamente).
    var n int = 42
    var f float64 = float64(n)
    fmt.Printf("%d -> %g\n", n, f)

    // Inferenza: j ha lo stesso tipo di k (int).
    var k int
    j := k
    fmt.Printf("tipo di j: %T\n", j)

    // Il tipo dedotto da una costante letterale dipende dalla sua forma.
    x := 3.14 // float64
    y := 'a'  // rune (alias di int32)
    fmt.Printf("%T %T\n", x, y)

    // Non si può sommare int e float64 senza conversione:
    //   fmt.Println(n + f)   // errore di compilazione: mismatched types
    fmt.Println(float64(n) + f)
}
```

```
true go 42 7 1.5 2.25
42 -> 42
tipo di j: int
float64 int32
84
```

3.2.1 Analisi del codice

- `float64(n)` converte il valore: omettere la conversione e scrivere `n + f` con `n` di tipo `int` e `f` di tipo `float64` produce l'errore di compilazione `invalid operation: n + f (mismatched types int and float64)`.
- `%T` stampa il tipo dedotto: `int` per `j`, `float64` per la costante `3.14` e `int32` per `'a'`.
- In Go il letterale `'a'` è una *rune*, un alias di `int32` che rappresenta un punto di codice Unicode. Per questo `%T` stampa `int32`.

Confronto — C, Python, Java

In **C** somme tra `int` e `double` funzionano grazie alle conversioni aritmetiche implicite; in **Java** sono implicite le conversioni *di allargamento* (`int`→`double`). In **Go** nessuna: la scelta evita perdite di precisione o di segno non volute, al prezzo di un `float64(…)` in più nel codice. In **Python** i tipi sono dinamici e `1 + 2.5` funziona perché è l'interprete a decidere a runtime.

Capitolo 4

Funzioni

4.1 Dichiarazione e parametri

Una funzione può avere zero o più argomenti, e anche in questo caso il **tipo segue il nome**: `func add(x int, y int) int`. Quando più parametri consecutivi hanno lo stesso tipo si può abbreviare: `func add(x, y int) int`.

Listato 4.1: `02_functions/functions.go`

```go
// functions.go - Funzioni: parametri, tipo dopo il nome, forma abbreviata
// (slide 13).
package main

import "fmt"

// Il tipo segue il nome del parametro; anche il tipo di ritorno è in coda.
func add(x int, y int) int {
    return x + y
}

// Forma abbreviata: parametri consecutivi dello stesso tipo.
func addShort(x, y int) int {
    return x + y
}

func main() {
    fmt.Println(add(3, 4))
    fmt.Println(addShort(10, 20))
}
```

```
7
30
```

Il tipo di ritorno compare dopo la lista dei parametri. Go è *call-by-value*: `x` e `y` sono copie degli argomenti (si veda la Sezione 6.1 per come modificare l'originale con un puntatore).

Confronto — C, Python, Java

Chi viene da C o Java è abituato a `int add(int x, int y)`, con il tipo *prima* del nome. Go lo inverte (`x int`) per due ragioni leggibili: le dichiarazioni si leggono da sinistra a destra («`x` è un `int`») e le forme complesse (puntatori a funzioni, slice di mappe, …) restano leggibili. In Python le annotazioni di tipo seguono il nome (`x: int`) ma sono facoltative e non verificate a runtime.

4.2 Risultati multipli

Una funzione può restituire **un numero qualsiasi di risultati**.

Listato 4.2: `02_functions/swap.go`

```go
// swap.go - Una funzione può restituire più risultati (slide 13-14).
package main

import "fmt"

func swap(x, y string) (string, string) {
    return y, x
}

func main() {
    // Assegnazione multipla: entrambi i risultati vengono "spacchettati".
    a, b := swap("hello", "world")
    fmt.Println(a, b)
}
```

```
world hello
```

L'assegnazione multipla `a, b := swap("hello", "world")` spacchetta i due valori restituiti. Il caso d'uso più importante è restituire un **risultato e un errore**: l'errore è per convenzione l'*ultimo* valore restituito e `nil` significa «nessun errore». La Sezione 7.3 approfondisce questo schema; qui lo si vede con `os.ReadFile`.

Listato 4.3: `02_functions/result_and_error.go`

```go
// result_and_error.go - Convenzione risultato + errore (slide 13, 15).
//
// La slide usa os.ReadFile("example.txt"). Qui il file viene prima creato in
// una directory temporanea, così l'esempio è autonomo e non dipende da file
// preesistenti; poi si prova anche un file inesistente per mostrare il ramo
// di errore. Il messaggio dell'errore dipende dal sistema operativo, quindi
// qui viene stampato solo il fatto che l'errore sia avvenuto.
package main

import (
    "fmt"
    "os"
    "path/filepath"
)

func readAndPrint(name string) {
    data, err := os.ReadFile(name)
    // Per convenzione l'errore è l'ultimo valore restituito e nil
    // significa "nessun errore". Va controllato subito.
    if err != nil {
        fmt.Println("Error: lettura fallita, err != nil")
        return
    }
    fmt.Println("File contents:", string(data))
}

```

```go
func main() {
    dir, err := os.MkdirTemp("", "goguide")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    defer os.RemoveAll(dir) // pulizia al termine di main

    path := filepath.Join(dir, "example.txt")
    if err := os.WriteFile(path, []byte("ciao dal file"), 0o644); err != nil {
        fmt.Println("Error:", err)
        return
    }

    readAndPrint(path)                          // caso di successo
    readAndPrint(filepath.Join(dir, "no.txt")) // caso di errore
}
```

```
File contents: ciao dal file
Error: lettura fallita, err != nil
```

Differenza rispetto alla slide

La slide legge un file `example.txt` che si assume già esistente. Per rendere l'esempio autonomo e riproducibile, il file viene prima creato in una directory temporanea (con `defer os.RemoveAll` per ripulirla), e viene provato anche un file inesistente per mostrare il ramo d'errore. La parte essenziale (`data, err := os.ReadFile(...)` seguito da `if err != nil`) è quella della slide. Il testo esatto dell'errore dipende dal sistema operativo, quindi viene stampato un messaggio fisso.

Confronto — C, Python, Java

In **C** per restituire più valori si usano parametri di output (puntatori) o struct; in **Python** si restituisce una tupla (meccanismo simile a quello di Go); in **Java** servirebbe un oggetto contenitore. Go ha il supporto nativo ai risultati multipli, ed è la base della gestione degli errori (niente eccezioni).

4.3 Valori di ritorno con nome

Go permette di dare un nome ai valori di ritorno nella firma. Le variabili con nome sono inizializzate al loro **valore zero** e un `return` senza argomenti (*«naked return»*) restituisce i loro valori correnti.

Listato 4.4: `02_functions/named_returns.go`

```go
// named_returns.go - Valori di ritorno con nome e "naked return" (slide 16).
package main

import "fmt"

// x e y sono variabili locali dichiarate nella firma, inizializzate al
// valore zero. Il "return" nudo restituisce i loro valori correnti.
func split(sum int) (x, y int) {
```

```
    x = sum * 4 / 9
    y = sum - x
    return // equivale a: return x, y
}

func main() {
    fmt.Println(split(17))
}
```

```
7 10
```

4.3.1 Analisi del codice

`x` e `y` sono variabili locali dichiarate nella firma `(x, y int)`. Con `sum = 17`: `x = 17 * 4 / 9 = 7` (divisione intera) e `y = 17 - 7 = 10`. Il `return` nudo equivale a `return x, y`. L'output `7 10` è quello stampato da `fmt.Println(split(17))`, in cui i due risultati diventano due argomenti.

Osservazione: naked return e leggibilità

Il *naked return* è comodo in funzioni brevi, ma in funzioni lunghe obbliga il lettore a risalire alla firma per sapere che cosa viene restituito. I nomi dei risultati hanno anche valore di documentazione: `(x, y int)` dice più di `(int, int)`.

4.4 Funzioni variadiche

Una **funzione variadica** accetta un numero variabile di argomenti. `fmt.Println` ne è un esempio. Si dichiara con il suffisso `...tipo` sull'ultimo parametro.

Listato 4.5: `02_functions/variadic.go`

```
// variadic.go - Funzioni variadiche (slide 68).
package main

import "fmt"

// Dentro la funzione "nums" è una slice []int.
func sum(nums ...int) {
    fmt.Print(nums, " ")
    total := 0
    for _, num := range nums {
        total += num
    }
    fmt.Println(total)
}

func main() {
    sum(1, 2)
    sum(1, 2, 3)

    // Per passare una slice esistente si usa il suffisso "...".
    nums := []int{1, 2, 3, 4}
    sum(nums...)
```

```
23
24      // Zero argomenti: nums è una slice nil e il totale è 0.
25      sum()
26  }
```

```
[1 2] 3
[1 2 3] 6
[1 2 3 4] 10
[] 0
```

4.4.1 Analisi del codice

- All'interno della funzione, `nums` è una **slice** `[]int` (Sezione 6.3): si può iterare con `range` e usare `len`.
- Per passare una slice già esistente come lista di argomenti si usa il suffisso `...` *alla chiamata*: `sum(nums...)`.
- Chiamata senza argomenti: `nums` è una slice `nil`, quindi stampa `[]` e il totale è `0`.

Confronto — C, Python, Java

In **C** le funzioni variadiche usano `stdarg.h` (`va_list`) e non c'è controllo di tipo; in **Java** la sintassi `int... nums` è praticamente identica a quella di Go; in **Python** si usa `*args`, e l'espansione `sum(*nums)` corrisponde a `sum(nums...)` in Go.

4.5 Funzioni anonime e closure

Le funzioni Go possono essere **anonime**, cioè senza nome: si definiscono *in linea* e si usano subito o si assegnano a una variabile.

Listato 4.6: `02_functions/anonymous.go`

```
1  // anonymous.go - Funzioni anonime (slide 37).
2  package main
3
4  import "fmt"
5
6  func main() {
7      // Funzione anonima assegnata a una variabile.
8      f := func(x, y int) int {
9          return x + y
10     }
11     fmt.Println(f(3, 4)) // 7
12
13     // Funzione anonima definita e invocata subito: le parentesi finali
14     // () eseguono la chiamata.
15     func(msg string) {
16         fmt.Println("invocata subito:", msg)
17     }("ciao")
18 }
```

```
7
invocata subito: ciao
```

Definizione — closure

Una **closure** è una funzione anonima annidata che *cattura le variabili* dell'ambiente che la racchiude. Mantiene un proprio **stato privato**, indipendente da quello di altre istanze.

Listato 4.7: `02_functions/closure.go`

```go
// closure.go - Closure con stato privato (slide 38).
package main

import "fmt"

// intSeq restituisce una funzione. La funzione anonima "cattura" la
// variabile i dell'ambiente che la racchiude: i sopravvive alle
// chiamate di intSeq ed è privata di quella particolare closure.
func intSeq() func() int {
    i := 0
    return func() int {
        i++
        return i
    }
}

func main() {
    nextInt := intSeq()
    fmt.Println(nextInt()) // 1
    fmt.Println(nextInt()) // 2
    fmt.Println(nextInt()) // 3

    // Una nuova closure ha una propria copia indipendente di i.
    newInts := intSeq()
    fmt.Println(newInts()) // 1
}
```

```
1
2
3
1
```

4.5.1 Analisi del codice

`intSeq()` dichiara `i := 0` e restituisce una funzione anonima che incrementa e restituisce `i`. Normalmente una variabile locale scompare quando la funzione ritorna; qui `i` *sopravvive* perché la closure lo referenzia (il runtime la sposta sullo heap, come per i puntatori a variabili locali, Sezione 6.1). Ogni chiamata di `intSeq()` crea una *nuova* variabile `i`: per questo `newInts()` ricomincia da `1` senza influenzare `nextInt`.

Un secondo esempio, elegante ed efficiente, genera i numeri di Fibonacci tenendo lo stato `(x, y)` nella closure:

Listato 4.8: `02_functions/closure_fib.go`

```go
// closure_fib.go - Fibonacci con una closure (slide 39).
package main

import "fmt"

// fibonacci restituisce una funzione che, a ogni chiamata, restituisce
// il numero di Fibonacci successivo. Lo stato (x, y) vive nella closure.
func fibonacci() func() int {
    x, y := 0, 1
    return func() int {
        // Assegnazione parallela: il lato destro è valutato per intero
        // prima di assegnare, quindi non serve una variabile temporanea.
        x, y = y, x+y
        return x
    }
}

func main() {
    f := fibonacci()
    for i := 0; i < 10; i++ {
        fmt.Println(f())
    }
}
```

```
1
1
2
3
5
8
13
21
34
55
```

L'assegnazione parallela `x, y = y, x+y` valuta *prima* tutto il lato destro e poi assegna, quindi non serve una variabile temporanea. A ogni chiamata, `x` avanza di un passo nella successione (1, 1, 2, 3, …).

Confronto — C, Python, Java

C non ha closure: si ottiene un effetto simile con un puntatore a funzione più un puntatore `void*` al «contesto», gestito a mano. In **Python** le closure esistono (per *modificare* una variabile catturata serve `nonlocal`). In **Java** le lambda catturano solo variabili *effectively final*: non possono modificarle, quindi serve di solito un oggetto contenitore (es. un array di un elemento o un `AtomicInteger`). In **Go** la closure cattura la variabile stessa, non una copia del suo valore, e può modificarla liberamente.

4.5.2 Closure come middleware

Un impiego pratico delle closure è il *middleware*: una closure può avvolgere una funzione *handler* per eseguire logica prima e/o dopo di essa. La slide mostra `timed`, che avvolge un

handler HTTP per misurare il tempo di elaborazione di una richiesta; con un server vero si registrerebbe con `http.HandleFunc("/hello", timed(hello))`.

Listato 4.9: `02_functions/middleware.go`

```go
// middleware.go - Closure come middleware (slide 40).
//
// La slide mostra timed(), che avvolge un handler HTTP per misurarne il tempo
// di elaborazione. Per rendere l'esempio eseguibile senza aprire porte di
// rete, la richiesta viene simulata con il package httptest.
package main

import (
    "fmt"
    "net/http"
    "net/http/httptest"
    "time"
)

// timed riceve un handler e ne restituisce uno nuovo che esegue codice
// "prima" e "dopo" la chiamata all'handler originale. f è catturato dalla
// closure.
func timed(f func(http.ResponseWriter, *http.Request)) func(http.ResponseWriter, *
    ↪ http.Request) {
    return func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        f(w, r)
        // Il tempo esatto cambia a ogni esecuzione: lo si arrotonda per
        // rendere l'output (quasi sempre) stabile.
        fmt.Println("The request took", time.Since(start).Round(50*time.
    ↪ Millisecond))
    }
}

func hello(w http.ResponseWriter, r *http.Request) {
    time.Sleep(100 * time.Millisecond) // lavoro simulato
    fmt.Fprintln(w, "hello")
}

func main() {
    // Con un server vero si scriverebbe: http.HandleFunc("/hello", timed(hello))
    handler := timed(hello)

    req := httptest.NewRequest("GET", "/hello", nil)
    rec := httptest.NewRecorder()
    handler(rec, req)

    fmt.Print("risposta: ", rec.Body.String())
}
```

```
The request took 100ms
risposta: hello
```

Come è stato reso eseguibile

La slide mostra solo `timed`. Per verificarlo senza aprire una porta di rete, l'esempio simula la richiesta con il package `httptest` della libreria standard. Il tempo viene arrotondato a multipli di 50 ms per rendere l'output quasi sempre stabile; il valore esatto, essendo una misura di tempo, non è deterministico: l'handler dorme 100 ms, quindi si legge circa `100ms`.

4.5.3 Funzioni anonime e goroutine: una nota sulle variabili di ciclo

Le closure sono usate molto spesso nel codice concorrente (Capitolo 8), dove una funzione anonima è lanciata come goroutine all'interno di un ciclo. Il punto delicato è se la closure veda una *variabile di ciclo condivisa* da tutte le iterazioni oppure una nuova per ciascuna. Dal Go 1.22, ogni iterazione di un ciclo `for i := …` ha la propria copia di `i`, quindi `go func() { use(i) }()` funziona come ci si aspetta; con versioni precedenti (o con `go.mod` che dichiara una versione anteriore) le closure condividevano un'unica `i`, causando bug sottili. Gli esempi di questa guida dichiarano `go 1.22` in `go.mod`.

Capitolo 5

Controllo di flusso

Le istruzioni di controllo del flusso di Go sono `for`, `if` (con `else`), `switch` e `defer`. Hanno in comune due regole di stile che le distinguono da C e Java: **nessuna parentesi** attorno alle condizioni e **graffe sempre obbligatorie**, anche per un solo statement.

5.1 Il ciclo `for`

Go ha **un solo costrutto di ciclo**: `for`. Nella sua forma completa ha tre componenti: l'istruzione *init*, l'espressione *condizione* e l'istruzione *post*.

Listato 5.1: Forma generale

```
for init; condizione; post {
    // corpo
}
```

Le istruzioni *init* e *post* sono facoltative: se omesse, il `for` diventa il «while» di Go. Se si omette anche la condizione, il ciclo è **infinito** (`for { }`).

Listato 5.2: `03_control_flow/for_loops.go`

```
1  // for_loops.go - L'unico costrutto di ciclo di Go: for (slide 20-21).
2  package main
3
4  import "fmt"
5
6  func main() {
7      // Forma completa: init; condizione; post. Niente parentesi attorno
8      // ai tre componenti, graffe sempre obbligatorie.
9      sum := 0
10     for i := 0; i < 10; i++ {
11         sum += i
12     }
13     fmt.Println("somma 0..9 =", sum)
14
15     // Senza init e post il for si comporta come un "while".
16     sum = 1
17     for sum < 1000 {
18         sum += sum
19     }
20     fmt.Println("primo valore >= 1000:", sum)
21
22     // Senza condizione il ciclo è infinito; si esce con break (o return).
23     n := 0
24     for {
25         n++
26         if n == 5 {
```

```
            break
        }
    }
    fmt.Println("uscito dal ciclo infinito con n =", n)
}
```

```
somma 0..9 = 45
primo valore >= 1000: 1024
uscito dal ciclo infinito con n = 5
```

5.1.1 Analisi del codice

- Il primo ciclo somma $0 + 1 + \cdots + 9 = 45$.
- Il secondo è un *while*: `sum` raddoppia a ogni iterazione (1, 2, 4, …) finché non raggiunge o supera 1000, cioè $2^{10} = 1024$.
- Il terzo è infinito e viene interrotto con `break` quando `n` vale 5.

Confronto — C, Python, Java

In **C** e **Java** esistono `for`, `while` e `do-while`; in **Python** `for … in` e `while`. Go li unifica in un solo `for`. L'iterazione su una collezione si ottiene con la clausola `range` (Sezione 6.5), che ricorda il `for-in` di Python.

5.1.2 Esempio: `echo`

Il programma `echo` stampa gli argomenti della riga di comando. `os.Args` è una *slice* di stringhe (Sezione 6.3); l'elemento di indice 0 è il nome del programma, quindi il ciclo parte da 1.

Listato 5.3: `03_control_flow/echo.go`

```
// Echo prints its command-line arguments. (slide 22)
//
// Esecuzione: go run echo.go Hello Go World
package main

import (
    "fmt"
    "os"
)

func main() {
    // s e sep sono inizializzate implicitamente alla stringa vuota.
    var s, sep string
    // os.Args è una slice di stringhe; os.Args[0] è il nome del programma,
    // quindi gli argomenti utili partono da 1.
    for i := 1; i < len(os.Args); i++ {
        // Al primo giro sep è "" (nessuno spazio iniziale); poi diventa " ".
        s += sep + os.Args[i]
        sep = " "
    }
    fmt.Println(s)
}
```

```
Hello Go World
```

Nell'esempio `s` e `sep` sono implicitamente inizializzate alla stringa vuota (valore zero). Al primo giro `sep` è vuoto e non si inserisce alcuno spazio iniziale; dal secondo giro `sep` vale `" "`. Il comando eseguito è `go run echo.go Hello Go World`.

5.2 Istruzioni condizionali: `if`

L'istruzione `if` (con `else`) ha forma simile a quella del `for`: nessuna parentesi attorno alla condizione, graffe sempre richieste, e si possono concatenare più condizioni con `if…else if…else`. Come nel `for`, c'è un'istruzione *opzionale* prima della condizione, separata da `;`:

Listato 5.4: Forma generale

```
if istruzioneOpzionale1; espressioneBooleana1 {
    blocco1
} else if istruzioneOpzionale2; espressioneBooleana2 {
    blocco2
} else {
    blocco3
}
```

Listato 5.5: `03_control_flow/if_else.go`

```
1  // if_else.go - if con istruzione di inizializzazione (slide 23-24).
2  package main
3
4  import (
5      "fmt"
6      "math"
7  )
8
9  // pow restituisce x^n se è inferiore a limit, altrimenti stampa un
10 // messaggio e restituisce limit.
11 func pow(x, n, limit float64) float64 {
12     // v è dichiarata nell'istruzione iniziale dell'if: è visibile nel blocco
13     // if e nel relativo else, ma non dopo.
14     if v := math.Pow(x, n); v < limit {
15         return v
16     } else {
17         // Nota: "} else {" deve stare sulla stessa riga.
18         fmt.Printf("%g >= %g\n", v, limit)
19     }
20     // Qui "v" non esiste più (errore di compilazione se la si usasse).
21     return limit
22 }
23
24 func main() {
25     fmt.Println(pow(3, 2, 10), pow(3, 3, 20))
26
27     // Catena if / else if / else, ciascuno con la propria istruzione
28     // opzionale.
29     x := 7
30     if r := x % 2; r == 0 {
```

```
31          fmt.Println("pari")
32      } else if r == 1 {
33          fmt.Println("dispari")
34      } else {
35          fmt.Println("negativo e dispari")
36      }
37  }
```

```
27 >= 20
9 20
dispari
```

5.2.1 Analisi del codice

- L'istruzione opzionale è usata spesso per una breve dichiarazione di variabile. In `if v := math.Pow(x, n); v < limit`, la variabile `v` esiste *solo* nel blocco `if` e nel relativo `else` (*scope*): dopo l'`if` non è più visibile.
- `pow(3, 2, 10)` calcola $3^2 = 9 < 10$ e restituisce 9. `pow(3, 3, 20)` calcola $27 \geq 20$: stampa `27 >= 20` (messaggio del ramo `else`) e restituisce il limite, 20. Poiché gli argomenti di `Println` sono valutati prima, la riga `27 >= 20` compare *prima* di `9 20`.
- L'`else` deve stare sulla stessa riga della graffa di chiusura (Sezione 2.6).
- L'ultimo blocco mostra una catena `else if` con l'istruzione `r := x % 2`.

5.3 Istruzioni condizionali: `switch`

L'istruzione `switch` sceglie *uno* tra molti casi da eseguire. I `case` sono valutati dall'alto verso il basso e ci si ferma al primo che corrisponde.

Le differenze rispetto a C sono sostanziali:

- viene eseguito **solo il caso selezionato**, non tutti quelli seguenti: il `break` di C è fornito *automaticamente*;
- i `case` **non devono essere costanti**;
- i valori possono essere **di qualunque tipo**, non solo interi (le slide usano stringhe).

Listato 5.6: `03_control_flow/switch.go`

```
1  // switch.go - switch senza fallthrough implicito (slide 25).
2  package main
3
4  import "fmt"
5
6  func describe(day string) {
7      switch day {
8      case "Monday":
9          fmt.Println("Start of the week")
10     case "Friday":
11         fmt.Println("Almost weekend!")
12     case "Saturday", "Sunday": // più valori nello stesso case
```

```
13          fmt.Println("Weekend!")
14      default:
15          fmt.Println("Giorno qualunque")
16      }
17  }
18
19  func main() {
20      for _, d := range []string{"Monday", "Friday", "Sunday", "Wednesday"} {
21          describe(d)
22      }
23
24      // I case non devono essere costanti: possono essere espressioni.
25      // Senza espressione dopo "switch" equivale a "switch true": viene
26      // eseguito il primo case la cui condizione è vera.
27      n := 15
28      switch {
29      case n < 10:
30          fmt.Println("piccolo")
31      case n < 100:
32          fmt.Println("medio")
33      default:
34          fmt.Println("grande")
35      }
36  }
```

```
Start of the week
Almost weekend!
Weekend!
Giorno qualunque
medio
```

5.3.1 Analisi del codice

`describe` confronta `day` con le stringhe dei `case`. `case "Saturday", "Sunday"` raggruppa più valori nello stesso ramo. Per `"Wednesday"` nessun caso corrisponde e viene eseguito `default`. Nell'ultima parte, `switch` *senza espressione* equivale a `switch true`: i `case` sono condizioni booleane arbitrarie e viene eseguito il primo che risulta vero (qui `n < 100` perché `n` vale 15).

Confronto — C, Python, Java

In **C** (e nel `switch` classico di **Java**) senza `break` l'esecuzione «cade» nel caso successivo: è una fonte tipica di errori. In **Go** il comportamento predefinito è l'opposto, quindi un caso non interferisce con il successivo. In C le etichette devono essere costanti intere; in Go possono essere espressioni di tipo arbitrario. In **Python** (3.10+) il costrutto equivalente è `match`.

5.4 L'istruzione `defer`

5.4.1 Introduzione

`defer` **rinvia l'esecuzione** di una chiamata di funzione *fino al momento in cui la funzione che la contiene ritorna*. Il problema che risolve è la pulizia delle risorse (chiusura di file e connessioni,

rilascio di lock): si scrive l'azione di rilascio *subito dopo* l'acquisizione, e il linguaggio garantisce che venga eseguita comunque la funzione termini (anche con un `return` anticipato).

5.4.2 Funzionamento

- Gli **argomenti** della chiamata differita sono valutati *immediatamente*, ma la chiamata effettiva avviene solo quando la funzione circostante è terminata.
- Le chiamate differite sono impilate in uno **stack** ed eseguite in ordine **LIFO**: l'ultima registrata è la prima eseguita.

Listato 5.7: `03_control_flow/defer.go`

```go
// defer.go - Esecuzione differita (slide 26).
package main

import "fmt"

func main() {
    // La chiamata viene registrata ora (argomenti valutati ora) ma eseguita
    // solo quando main sta per terminare.
    defer fmt.Println("world")
    fmt.Println("hello")
}
```

```
hello
world
```

Il secondo esempio mostra le due proprietà contemporaneamente: l'ordine LIFO e la valutazione immediata degli argomenti.

Listato 5.8: `03_control_flow/defer_lifo.go`

```go
// defer_lifo.go - Ordine LIFO e valutazione immediata degli argomenti
// (slide 26).
package main

import "fmt"

func countdown() {
    // Le chiamate differite sono impilate: l'ultima registrata è la prima
    // eseguita. Ogni defer cattura il valore corrente di i (argomento
    // valutato subito).
    for i := 1; i <= 3; i++ {
        defer fmt.Println("defer", i)
    }
    fmt.Println("fine del corpo di countdown")
}

func main() {
    x := 1
    // L'argomento x viene valutato qui: la stampa mostrerà 1, non 2.
    defer fmt.Println("x al momento del defer =", x)
    x = 2
    fmt.Println("x ora =", x)
```

```
23
24      countdown()
25  }
```

```
x ora = 2
fine del corpo di countdown
defer 3
defer 2
defer 1
x al momento del defer = 1
```

5.4.3 Analisi del codice

Dentro `countdown`, il ciclo registra tre chiamate differite, con `i` uguale a 1, 2, 3 *valutato al momento del defer*. Il corpo stampa «fine del corpo di countdown»; solo *allo scadere della funzione* le chiamate sono eseguite in ordine inverso (3, 2, 1). In `main`, il `defer` che stampa `x` è registrato quando `x` vale 1; anche se poi `x` diventa 2, la riga differita stampa 1, perché l'argomento era già stato valutato. Essendo la prima registrata in `main`, è l'ultima a essere eseguita: compare in fondo all'output.

Confronto — C, Python, Java

In **Java** la pulizia si fa con `try/finally` (o `try-with-resources`), in **Python** con `try/finally` o `with`. In **C** tipicamente con un'etichetta di `goto cleanup`. `defer` offre il vantaggio di mettere il rilascio vicino all'acquisizione senza annidare blocchi. Si noti che `defer` si lega alla *funzione*, non al blocco: in un ciclo, i defer si accumulano fino alla fine della funzione.

Errore comune: defer dentro un ciclo

Poiché `defer` è eseguito solo all'uscita della *funzione*, scrivere `defer f.Close()` dentro un ciclo che apre molti file li tiene tutti aperti fino alla fine della funzione. In quei casi conviene racchiudere il corpo del ciclo in una funzione ausiliaria, in modo che il defer scatti a ogni iterazione.

Capitolo 6

Puntatori e strutture dati

6.1 Puntatori

6.1.1 Introduzione

Definizione — puntatore

Un **puntatore** è un valore che memorizza l'*indirizzo* di una variabile. Gli operatori sono quelli usuali: `&` produce l'indirizzo di una variabile; `*` *dereferenzia*, cioè recupera il valore a cui il puntatore punta.

Poiché Go è *call-by-value*, ogni argomento viene copiato: per permettere a una funzione di modificare una variabile del chiamante occorre passarne l'indirizzo.

6.1.2 Funzionamento e differenze rispetto a C

- **Nessuna aritmetica dei puntatori**: `p++` o `p+1` non compilano (errore reale: `invalid operation: p++ (non-numeric type *int)`). Questo elimina un'intera classe di errori di accesso alla memoria.
- Il **valore zero** di un puntatore è `nil`.
- È **sicuro restituire l'indirizzo di una variabile locale**: il runtime di Go fa sì che la variabile sopravviva alla funzione (se *«scappa»* dallo scope, viene allocata sullo *heap* anziché sullo stack).

Listato 6.1: `04_data_structures/pointers.go`

```go
// pointers.go - Puntatori (slide 27).
package main

import "fmt"

// Go è call-by-value: la funzione riceve una copia del puntatore, che però
// punta alla stessa variabile del chiamante.
func incrementByValue(n int) { n++ }
func incrementByPointer(n *int) {
    *n++ // dereferenziazione: modifica la variabile originale
}

// È sicuro restituire l'indirizzo di una variabile locale: il runtime la
// alloca sullo heap (escape) e il garbage collector la libera quando
// nessuno la referenzia più. In C sarebbe un dangling pointer.
func newCounter() *int {
    c := 100
    return &c
```

```
}

func main() {
    var p *int
    fmt.Println("valore zero del puntatore:", p) // <nil>

    i := 1
    p = &i          // p punta a i
    fmt.Println(*p) // 1
    *p = 2          // equivale a i = 2
    fmt.Println(i)  // 2

    incrementByValue(i)
    fmt.Println("dopo incrementByValue:", i) // invariato
    incrementByPointer(&i)
    fmt.Println("dopo incrementByPointer:", i)

    q := newCounter()
    *q++
    fmt.Println("contatore sullo heap:", *q)

    // Nessuna aritmetica dei puntatori: p++ o p+1 non compilano.
}
```

```
valore zero del puntatore: <nil>
1
2
dopo incrementByValue: 2
dopo incrementByPointer: 3
contatore sullo heap: 101
```

6.1.3 Analisi del codice

- `p = &i` fa puntare `p` a `i`; `*p = 2` equivale a `i = 2`.
- `incrementByValue(i)` riceve una *copia* di `i` e la incrementa: il chiamante non vede alcun cambiamento (resta 2). `incrementByPointer(&i)` riceve una copia del *puntatore*, che però punta alla stessa variabile: `*n++` la modifica (da 2 a 3).
- `newCounter` restituisce `&c` dove `c` è locale: è lecito perché `c` viene portata sullo heap. In C lo stesso codice restituirebbe un *dangling pointer* verso un frame di stack già distrutto.

Errore comune: dereferenziare un puntatore `nil`

Un puntatore non inizializzato vale `nil`; dereferenziarlo causa un *panic* a runtime: `panic: runtime error: invalid memory address or nil pointer dereference`. Come in C, il controllo `if p != nil` va fatto dove `nil` è un valore possibile; a differenza di C il programma non prosegue con memoria corrotta ma si arresta in modo controllato.

Confronto — C, Python, Java

Rispetto a **C**: stessa notazione `&` e `*`, ma senza aritmetica e con garbage collector (niente `free`). Rispetto a **Java** e **Python**, dove gli oggetti sono sempre referenziati implicitamente e la distinzione tra valore e riferimento è decisa dal tipo, in Go è *il programmatore a*

scegliere: un `Vertex` passato per valore viene copiato, un `*Vertex` no.

6.2 Strutture dati aggregate

6.2.1 Struct

Una **struct** è una collezione tipizzata di *campi*, di dimensione fissa, con sintassi simile a quella del C. I campi si accedono con la notazione `.`; si possono accedere anche tramite un puntatore a struct (`p := &v`), nel qual caso Go permette di scrivere `p.X` al posto di `(*p).X`.

Listato 6.2: `04_data_structures/structs.go`

```go
// structs.go - Struct (slide 28).
package main

import "fmt"

type Vertex struct {
    X int
    Y int
}

func main() {
    // Letterale con campi nominati; i campi omessi valgono zero.
    v := Vertex{X: 1, Y: 2}
    fmt.Println(v.X, v)

    // Un puntatore a struct: Go consente p.X al posto di (*p).X.
    p := &v
    p.X = 10
    fmt.Println(v.X) // 10: p punta a v

    // Le struct sono valori: l'assegnazione copia tutti i campi.
    w := v
    w.Y = 99
    fmt.Println(v, w) // v non cambia

    var zero Vertex
    fmt.Println(zero) // {0 0}
}
```

```
1 {1 2}
10
{10 2} {10 99}
{0 0}
```

Analisi del codice

`Vertex{X: 1, Y: 2}` è un *letterale composito* con campi nominati; quelli omessi valgono zero (`{0 0}` per `zero`). Dopo `p := &v` l'assegnazione `p.X = 10` modifica `v`, perché `p` punta a essa. Al contrario `w := v` **copia** tutti i campi: modificare `w.Y` lascia `v` invariata.

Confronto — C, Python, Java

La struct di Go è simile a quella di **C**: un valore, copiato nelle assegnazioni. A differenza di una *classe* Java o Python, la struct non è un riferimento, non ha ereditarietà e i metodi si associano separatamente (Capitolo 7).

6.2.2 Array

Un **array** `[n]T` è un array di `n` elementi di tipo `T`, di **dimensione fissa** (non ridimensionabile). La lunghezza fa parte del tipo: `[2]string` e `[3]string` sono tipi *diversi*.

Listato 6.3: `04_data_structures/arrays.go`

```go
// arrays.go - Array a dimensione fissa (slide 28).
package main

import "fmt"

func main() {
    // La lunghezza fa parte del tipo: [2]string e [3]string sono tipi
    // diversi.
    var a [2]string
    a[0] = "Hello"
    a[1] = "World"
    fmt.Println(a[0], a[1])
    fmt.Println(a)

    primes := [6]int{2, 3, 5, 7, 11, 13}
    fmt.Println(primes, len(primes))

    // Un array è un valore: l'assegnazione ne copia tutti gli elementi
    // (diverso da C, dove un array decade in puntatore).
    b := primes
    b[0] = 99
    fmt.Println(primes[0], b[0]) // 2 99
}
```

```
Hello World
[Hello World]
[2 3 5 7 11 13] 6
2 99
```

Confronto — C, Python, Java

In **C** un array nelle espressioni «decade» in un puntatore al primo elemento: passarlo a una funzione non lo copia. In **Go** un array è un *valore*: assegnarlo o passarlo copia tutti gli elementi (come mostra `b := primes`). In **Java** gli array sono oggetti referenziati. Per questo in Go gli array si usano di rado direttamente: si usano le *slice*.

6.3 Slice

6.3.1 Introduzione

La **slice** è il tipo di dato più importante di Go e molto più flessibile dell'array. `[]T` è una slice con elementi di tipo `T`: una *vista*, di dimensione dinamica, su un **array sottostante**.

6.3.2 Creare una slice da un array

Si crea una slice *affettando* un array (o un'altra slice) con due indici, `s[i:j]`: il limite inferiore è **incluso**, quello superiore **escluso**. Una slice *riferisce* una sezione dell'array: modificare un elemento della slice modifica l'elemento corrispondente dell'array.

Listato 6.4: `04_data_structures/slices.go`

```go
// slices.go - Slice come vista su un array sottostante (slide 29-30).
package main

import "fmt"

func main() {
    primes := [6]int{2, 3, 5, 7, 11, 13}

    // s[i:j] include l'elemento i ed esclude j.
    var s []int = primes[1:4]
    fmt.Println(s, "len =", len(s), "cap =", cap(s)) // [3 5 7] len=3 cap=5

    // La slice NON copia i dati: modificare s modifica l'array.
    s[0] = 100
    fmt.Println("array dopo s[0]=100:", primes)

    // Due slice sulla stessa porzione si influenzano a vicenda.
    t := primes[:2]
    fmt.Println("t =", t) // [2 100]

    // La capacità arriva fino alla fine dell'array sottostante.
    u := s[:5]
    fmt.Println(u, len(u), cap(u))

    // Controllo dei limiti a runtime: fuori dalla capacità -> panic.
    defer func() {
        fmt.Println("recuperato:", recover())
    }()
    _ = s[:6] // slice bounds out of range
}
```

```
[3 5 7] len = 3 cap = 5
array dopo s[0]=100: [2 100 5 7 11 13]
t = [2 100]
[100 5 7 11 13] 5 5
recuperato: runtime error: slice bounds out of range [:6] with capacity 5
```

6.3.3 Analisi del codice

- `primes[1:4]` contiene gli indici 1, 2, 3, cioè `[3 5 7]` (come nella slide): `len = 3`, `cap = 5`.
- `s[0] = 100` modifica `primes[1]`: la slice non contiene una copia dei dati. La slice `t := primes[:2]`, definita dopo, vede il nuovo valore (`[2 100]`).
- `s[:5]` è consentito perché la *capacità* di `s` è 5 (fino alla fine dell'array sottostante), anche se la lunghezza è 3.
- `s[:6]` eccede la capacità: Go controlla i limiti e causa un panic, qui intercettato con `recover` dentro un `defer` solo per mostrare il messaggio `slice bounds out of range [:6] with capacity 5`.

Come è fatta una slice

Una slice è un piccolo descrittore con tre campi: un **puntatore** all'array sottostante, la **lunghezza** (`len`) e la **capacità** (`cap`). Copiare una slice copia questo descrittore, non gli elementi. Ecco perché due slice sullo stesso array si influenzano a vicenda e perché passare una slice a una funzione è economico.

Confronto — C, Python, Java

In **Python** `a[1:4]` crea una *copia* della sezione; in Go la slice *condivide* la memoria. In **Java** il costrutto più simile a una slice è la vista `List.subList`, mentre un `ArrayList` somiglia alla slice per la crescita dinamica. In **C** si ottiene qualcosa di simile con la coppia (puntatore, lunghezza), ma a mano e senza controlli.

6.3.4 Lunghezza, capacità e `make`

Una slice ha sia una **lunghezza** sia una **capacità**. `len(s)` è il numero di elementi che contiene; `cap(s)` è il numero di elementi dell'array sottostante a partire dall'inizio della slice. Go effettua il *bounds checking*: l'accesso fuori dai limiti causa un errore di compilazione (se riconoscibile staticamente) o di runtime.

Una slice si può creare anche con `make`, che permette di specificare lunghezza e (facoltativamente) capacità: `s := make([]int, 5, 10)`. L'array sottostante viene creato automaticamente, ma non è direttamente visibile.

Listato 6.5: `04_data_structures/making_slice.go`

```go
// making_slice.go - Creare slice con make (slide 30-31).
package main

import "fmt"

func main() {
    a := make([]int, 0, 5) // len(s)=0, cap(s)=5
    printSlice("a", a)

    b := make([]int, 5) // len=5, cap=5: cinque zeri
    printSlice("b", b)

    c := make([]int, 5, 10)
    printSlice("c", c)
```

```
15
16      d := c[2:4] // len=2, cap=8 (si parte dall'indice 2 dell'array)
17      printSlice("d", d)
18
19      var nilSlice []int // slice nil: len=0, cap=0, ma utilizzabile con append
20      printSlice("nil", nilSlice)
21      fmt.Println("nilSlice == nil?", nilSlice == nil)
22  }
23
24  func printSlice(s string, x []int) {
25      fmt.Printf("%s len=%d cap=%d %v\n", s, len(x), cap(x), x)
26  }
```

```
a len=0 cap=5 []
b len=5 cap=5 [0 0 0 0 0]
c len=5 cap=10 [0 0 0 0 0]
d len=2 cap=8 [0 0]
nil len=0 cap=0 []
nilSlice == nil? true
```

Analisi del codice

`make([]int, 0, 5)` crea una slice vuota (`len=0`) con spazio per 5 elementi (`cap=5`): è il primo output, identico a quello della slide (`a len=0 cap=5 []`). Con `make([]int, 5)` la capacità coincide con la lunghezza e gli elementi sono cinque zeri. `d := c[2:4]` ha `len=2` e `cap=8`: la capacità si conta dall'inizio della slice (indice 2) fino alla fine dell'array (10). Infine una slice dichiarata e non inizializzata è `nil`: ha `len=0`, `cap=0` ed è comunque utilizzabile con `append`.

Errore comune: confondere array e slice

`[3]int` è un array (tipo con lunghezza, *valore*); `[]int` è una slice (tipo senza lunghezza, *vista*). Un array assegnato si copia, una slice assegnata condivide l'array sottostante. Un indice costante fuori limite su un array è rifiutato dal compilatore (`invalid argument: index 5 out of bounds [0:3]`), mentre un indice variabile fuori limite dà un panic a runtime (`index out of range [5] with length 3`).

6.3.5 L'operazione `append`

Si aggiungono elementi a una slice con `append`, la cui firma concettuale è `func append(slice []T, elems ...T) []T`. Quando si aggiunge a una slice, la slice *può essere ingrandita* se necessario.

- Se `len(s) < cap(s)`: i nuovi elementi sono aggiunti all'**array sottostante esistente**, senza crearne uno nuovo.
- Se `len(s) == cap(s)`: Go **alloca un nuovo array più grande** (di norma il doppio della capacità precedente), **copia** gli elementi esistenti, e **restituisce una slice** che punta al nuovo array. Il vecchio array resta invariato se altre slice vi puntano ancora.

Listato 6.6: `04_data_structures/append.go`

```
1  // append.go - append e crescita della capacità (slide 32-33).
```

```go
package main

import "fmt"

func main() {
    var s []int
    printSlice(s)

    s = append(s, 0) // funziona su slice nil
    printSlice(s)

    s = append(s, 1) // la slice cresce quando serve
    printSlice(s)

    s = append(s, 2, 3, 4) // più elementi in una volta
    printSlice(s)

    // Capacità piena: viene allocato un nuovo array e gli elementi sono
    // copiati.
    t := make([]int, 3, 3)
    t = append(t, 4)
    fmt.Println(t) // [0 0 0 4]
    printSlice(t)

    // Capacità disponibile: nessuna nuova allocazione, e l'array
    // sottostante è condiviso.
    base := make([]int, 3, 10)
    x := append(base, 1)
    y := append(base, 2)    // sovrascrive la stessa posizione di x!
    fmt.Println(x[3], y[3]) // 2 2
}

func printSlice(s []int) {
    fmt.Printf("len=%d cap=%d %v\n", len(s), cap(s), s)
}
```

```
len=0 cap=0 []
len=1 cap=1 [0]
len=2 cap=2 [0 1]
len=5 cap=6 [0 1 2 3 4]
[0 0 0 4]
len=4 cap=6 [0 0 0 4]
2 2
```

6.3.6 Analisi del codice

- `append` funziona anche su una slice `nil`, come mostrano le prime righe.
- `s = append(s, 2, 3, 4)` aggiunge tre elementi in una chiamata. Dopo l'aggiunta la lunghezza è 5, ma la capacità è **6**, non 5. La slide parla di «di norma il doppio»; la regola esatta è un dettaglio dell'implementazione: il runtime calcola la nuova capacità e poi la *arrotonda* alla dimensione dei blocchi che l'allocatore fornisce. Per questo il valore osservato (6) può non coincidere con una semplice regola «raddoppia»: qui servivano 5 posti, più del doppio della capacità precedente (2×2), e l'arrotondamento ha portato a 6. Ciò che resta valido è la proprietà qualitativa: la capacità cresce a salti, non a ogni

elemento, e ogni salto copia l'array. La regola esatta è un dettaglio dell'implementazione e non va data per scontata.

- Il caso della slide, `make([]int, 3, 3)` seguito da `append(s, 4)`, dà `[0 0 0 4]`: la capacità era piena, quindi è stato creato un nuovo array.
- Nell'ultimo blocco, `base` ha capacità 10 e lunghezza 3: entrambe le chiamate `append(base, …)` scrivono nella *stessa* posizione dell'array condiviso (indice 3), quindi `x[3]` e `y[3]` valgono entrambi 2 — la seconda chiamata ha sovrascritto la prima.

Errore comune: dimenticare di assegnare il risultato di `append`

`append` può restituire una slice che punta a un array *nuovo*, quindi il risultato **va sempre riassegnato**. Scrivere `append(s, 4)` senza riassegnare è rifiutato dal compilatore (`append(s, 2) (value of type []int) is not used`). Un errore più subdolo si ha passando la slice a una funzione che fa `append`: la funzione riceve una *copia* del descrittore, quindi il chiamante non vede né la nuova lunghezza né il nuovo array.

Listato 6.7: `04_data_structures/slice_pitfalls.go`

```go
// slice_pitfalls.go - Errori comuni con array e slice.
package main

import "fmt"

// Errato: append può restituire una slice con un nuovo array; il
// risultato va riassegnato. Qui la modifica si perde per il chiamante,
// perché s è una copia dell'header (puntatore, len, cap).
func addWrong(s []int) {
    s = append(s, 99)
}

// Corretto: restituire la slice aggiornata.
func addRight(s []int) []int {
    return append(s, 99)
}

func main() {
    s := []int{1, 2, 3}
    addWrong(s)
    fmt.Println("dopo addWrong:", s) // [1 2 3]
    s = addRight(s)
    fmt.Println("dopo addRight:", s) // [1 2 3 99]

    // append senza riassegnare: il compilatore stesso lo rifiuta
    // ("append(s, 4) (value of type []int) is not used"). Va scritto
    // s = append(s, 4).

    // Una funzione può però modificare gli ELEMENTI esistenti, perché la
    // slice condivide l'array sottostante.
    double(s)
    fmt.Println("dopo double:", s)

    // Array e slice sono diversi: l'array si copia per valore.
    arr := [3]int{1, 2, 3}
    arrCopy := arr
    arrCopy[0] = 100
```

```
    fmt.Println(arr, arrCopy)
}

func double(s []int) {
    for i := range s {
        s[i] *= 2
    }
}
```

```
dopo addWrong: [1 2 3]
dopo addRight: [1 2 3 99]
dopo double: [2 4 6 198]
[1 2 3] [100 2 3]
```

Perché avviene

`addWrong` riceve una copia di `(puntatore, len, cap)` e ne aggiorna la *copia*: la `len` del chiamante resta 3. `addRight` restituisce la slice aggiornata, che il chiamante riassegna. La funzione `double` mostra invece che modificare gli *elementi* funziona, perché tutte le copie del descrittore puntano allo stesso array.

6.4 Map

6.4.1 Introduzione e funzionamento

Una **map** è una collezione di coppie chiave–valore. Il tipo `map[K]V` è un *riferimento* a una tabella hash in cui `K` e `V` sono i tipi di chiavi e valori. Si crea con `make`: `m = make(map[string]Vertex)`.

Le operazioni principali sono:

```
m[key] = element      // inserimento o aggiornamento
elem := m[key]        // lettura
delete(m, key)        // cancellazione
elem, ok = m[key]     // test: ok è true se la chiave esiste
```

Listato 6.8: `04_data_structures/maps.go`

```
// maps.go - Map: tabelle hash chiave-valore (slide 34).
package main

import "fmt"

type Vertex struct {
    Lat, Long float64
}

func main() {
    // Una map va creata con make (o con un letterale): la map nil
    // permette letture ma non scritture.
    m := make(map[string]Vertex)
    m["Bell Labs"] = Vertex{40.68433, -74.39967} // inserimento
```

```go
    fmt.Println(m["Bell Labs"])                    // lettura

    m["Bell Labs"] = Vertex{1, 2} // aggiornamento
    fmt.Println(m)

    // Chiave inesistente: si ottiene il valore zero, senza errore.
    fmt.Println("chiave assente:", m["Google"])

    // Idioma "comma ok": distingue "assente" da "presente con valore zero".
    elem, ok := m["Google"]
    fmt.Println(elem, ok) // {0 0} false
    elem, ok = m["Bell Labs"]
    fmt.Println(elem, ok) // {1 2} true

    delete(m, "Bell Labs") // cancellazione (nessun errore se assente)
    fmt.Println(len(m))

    // Errore comune: scrivere su una map nil provoca panic.
    var nilMap map[string]int
    fmt.Println(nilMap["x"]) // lettura ok: 0
    defer func() { fmt.Println("recuperato:", recover()) }()
    nilMap["x"] = 1 // panic: assignment to entry in nil map
}
```

```
{40.68433 -74.39967}
map[Bell Labs:{1 2}]
chiave assente: {0 0}
{0 0} false
{1 2} true
0
0
recuperato: assignment to entry in nil map
```

6.4.2 Analisi del codice

- Leggere una chiave *inesistente* **non è un errore**: si ottiene il *valore zero* del tipo dei valori (qui `{0 0}`).
- Per distinguere «chiave assente» da «chiave presente con valore zero» si usa l'idioma *comma ok*: `elem, ok := m[key]`.
- `delete` su una chiave assente non fa nulla.
- Scrivere su una map `nil` causa un panic (`assignment to entry in nil map`); la lettura da una map `nil` è invece lecita e restituisce il valore zero.

Errore comune: accesso a una chiave inesistente

Poiché la lettura di una chiave assente restituisce silenziosamente il valore zero, un contatore `m["x"]` sembra funzionare ma una mappa di booleani non distingue «falso» da «assente»: serve `v, ok := m[k]`. Ricordare inoltre di *creare* la map con `make` (o con un letterale) prima di scrivere.

Confronto — C, Python, Java

In **Python** una chiave assente solleva `KeyError` (o si usa `.get`); in **Java** `HashMap.get` restituisce `null`; in **Go** si ottiene il valore zero e si usa `ok` per sapere se la chiave c'era. In **C** non esiste una map nella libreria standard. Una differenza da non trascurare: l'ordine di iterazione di una map Go *non è specificato* (è anzi deliberatamente casuale), mentre un `dict` Python mantiene l'ordine di inserimento.

6.5 L'istruzione `range`

`range` itera sugli elementi di varie strutture dati (array, slice, stringhe, map e channel); il comportamento *dipende dal tipo*:

- array, slice, stringhe: `range` fornisce **indice e valore** di ogni elemento;
- map: fornisce **chiave e valore**;
- channel: fornisce i valori ricevuti finché il channel non è chiuso (Sezione 8.5.2).

Listato 6.9: `04_data_structures/range.go`

```
// range.go - Iterare con range su una slice (slide 35).
package main

import "fmt"

var pow = []int{1, 2, 4, 8, 16, 32, 64, 128}

func main() {
    // Per array e slice, range produce indice e copia del valore.
    for i, v := range pow {
        fmt.Printf("2**%d = %d\n", i, v)
    }
}
```

```
2**0 = 1
2**1 = 2
2**2 = 4
2**3 = 8
2**4 = 16
2**5 = 32
2**6 = 64
2**7 = 128
```

L'esempio successivo mostra come **ignorare** l'indice o il valore con il *blank identifier* `_`, e come iterare su una map.

Listato 6.10: `04_data_structures/range2.go`

```
// range2.go - range: ignorare indice o valore, iterare su map (slide 36).
package main

import "fmt"

```

```go
func main() {
    nums := []int{2, 3, 4}
    sum := 0
    // "_" (blank identifier) scarta l'indice: Go vieta variabili inutilizzate.
    for _, num := range nums {
        sum += num
    }
    fmt.Println("sum:", sum)

    for i, num := range nums {
        if num == 3 {
            fmt.Println("index:", i)
        }
    }

    // Su una map range produce chiave e valore. L'ordine di iterazione NON
    // è specificato (in Go è volutamente casuale): un ciclo diretto
    // "for k, v := range kvs" stampa le due coppie in ordine variabile.
    // Per avere un output stabile si itera su un elenco di chiavi
    // ordinato a mano e si consulta la map.
    kvs := map[string]string{"a": "apple", "b": "banana"}
    for _, k := range []string{"a", "b"} {
        fmt.Printf("%s -> %s\n", k, kvs[k])
    }

    // Con la sola chiave si omette il secondo valore.
    n := 0
    for k := range kvs {
        _ = k
        n++
    }
    fmt.Println("chiavi iterate:", n)

    // Anche le stringhe: range restituisce indice di byte e rune (Unicode).
    for i, r := range "gò" {
        fmt.Printf("%d:%c ", i, r)
    }
    fmt.Println()
}
```

```
sum: 9
index: 1
a -> apple
b -> banana
chiavi iterate: 2
0:g 1:ò
```

6.5.1 Analisi del codice

- `for _, num := range nums` scarta l'indice; Go vieta le variabili dichiarate e inutilizzate, quindi `_` è la forma corretta per dire «non mi serve».
- `for k := range kvs` itera sulle sole chiavi (valore omesso).
- **Ordine delle map:** la slide mostra l'output `a -> apple`, `b -> banana`; ma con un ciclo `for k, v := range kvs` diretto le due coppie possono comparire in *qualunque ordine*,

anche diverso da un'esecuzione all'altra. Per questo l'esempio della guida scorre un elenco di chiavi ordinato a mano e consulta la map: l'output è deterministico.

- Su una stringa `range` restituisce la posizione in *byte* e la *rune* (punto di codice Unicode): per `"gò"` gli indici sono 0 e 1 e il carattere `ò` occupa due byte UTF-8.

Capitolo 7

Metodi, interfacce ed errori

Go **non ha classi**. Il comportamento si associa ai tipi con i *metodi*, e le astrazioni si esprimono con le *interfacce*, che descrivono ciò che un tipo *sa fare* e non ciò che *contiene*.

7.1 Metodi

7.1.1 Introduzione e sintassi

Go supporta i **metodi** sui tipi concreti, comprese le struct.

Definizione — metodo e receiver

Un **metodo** è una funzione con un argomento speciale, il *receiver* (ricevitore): un parametro aggiuntivo scritto *tra* la parola chiave `func` e il nome del metodo. Associa la funzione a un tipo specifico (per esempio una struct) e permette di definire il comportamento del tipo.

Listato 7.1: Dalla slide: un metodo `Abs` su `Vertex`

```
type Vertex struct {
    X, Y float64
}

func (v Vertex) Abs() float64 {
    return math.Sqrt(v.X*v.X + v.Y*v.Y)
}
```

7.1.2 Funzionamento: receiver per valore e per puntatore

Il receiver è, a tutti gli effetti, un *parametro*: poiché Go passa tutto *per valore* (Capitolo 4), un receiver di tipo `Vertex` riceve una **copia** della struct. Se invece il receiver è un puntatore (`v *Vertex`) il metodo lavora sull'originale e può modificarlo. Le slide usano la forma a puntatore negli esempi sul *mutex* (`func (l *Lock) Lock()`, Capitolo 8), dove è indispensabile.

Listato 7.2: `05_methods_interfaces/methods.go`

```
1 // methods.go - Metodi su tipi concreti (slide 41).
2 package main
3
4 import (
5     "fmt"
6     "math"
7 )
8
```

```go
type Vertex struct {
    X, Y float64
}

// Un metodo è una funzione con un receiver, scritto tra "func" e il nome.
// Con receiver per valore (v Vertex) il metodo lavora su una COPIA.
func (v Vertex) Abs() float64 {
    return math.Sqrt(v.X*v.X + v.Y*v.Y)
}

// Con receiver puntatore (v *Vertex) il metodo può modificare l'originale.
func (v *Vertex) Scale(f float64) {
    v.X *= f
    v.Y *= f
}

// Errore comune: receiver per valore, la modifica agisce sulla copia
// e va persa.
func (v Vertex) ScaleWrong(f float64) {
    v.X *= f
    v.Y *= f
}

func main() {
    v := Vertex{3, 4}
    fmt.Println(v.Abs()) // 5

    v.ScaleWrong(10)
    fmt.Println("dopo ScaleWrong:", v) // invariato

    // v è una variabile indirizzabile: Go traduce v.Scale(10) in
    // (&v).Scale(10).
    v.Scale(10)
    fmt.Println("dopo Scale:", v, v.Abs())
}
```

```
5
dopo ScaleWrong: {3 4}
dopo Scale: {30 40} 50
```

7.1.3 Analisi del codice

`Abs` usa un receiver per valore: legge `X` e `Y` senza modificarli e restituisce $\sqrt{3^2 + 4^2} = 5$. `ScaleWrong` ha lo stesso receiver per valore ma tenta di modificare i campi: poiché agisce su una copia, `v` nel `main` rimane `{3 4}`. `Scale`, con receiver `*Vertex`, modifica l'originale: dopo `v.Scale(10)` il vettore vale `{30 40}` e `Abs()` dà 50. Si noti che nel `main` si scrive semplicemente `v.Scale(10)`: poiché `v` è una variabile, Go la traduce automaticamente in `(&v).Scale(10)`.

Errore comune: modificare con un receiver per valore

Dimenticare l'asterisco nel receiver di un metodo che deve cambiare lo stato è un errore classico, spesso silenzioso: il codice compila e funziona, ma le modifiche vanno perse con la copia. La regola pratica: se il metodo deve *modificare* il valore, serve un receiver puntatore.

Confronto — C, Python, Java

In **Java** e **Python** un metodo riceve il riferimento all'oggetto (`this`/`self`) e può sempre modificarne lo stato. In **Go** la scelta tra receiver per valore e per puntatore è esplicita e cambia la semantica. Rispetto a **C**, un metodo Go è «zucchero sintattico» per una funzione che prende la struct come primo parametro: `v.Abs()` corrisponde grosso modo a `Vertex_Abs(v)`. Il receiver è dichiarato *fuori* dalla definizione del tipo: i metodi non vivono dentro un blocco `class`.

7.2 Interfacce

7.2.1 Introduzione

Definizione — interfaccia

Un'**interfaccia** è una collezione *nominata* di firme di metodi.

Il punto più importante è l'**implementazione implicita**: in Go un tipo *soddisfa* un'interfaccia semplicemente implementandone *tutti* i metodi. Non serve alcuna dichiarazione esplicita (nessun `implements`): il tipo soddisfa automaticamente l'interfaccia.

L'obiettivo è progettare astrazioni basate sul **comportamento** (le azioni) e non sui dati. Le interfacce abilitano il **polimorfismo** in Go.

7.2.2 Esempio pratico: le forme geometriche

Listato 7.3: `05_methods_interfaces/interface.go`

```go
// interface.go - Interfacce implicite e polimorfismo (slide 42-45).
package main

import (
    "fmt"
    "math"
)

// Define the interface for geometric shapes
type geometry interface {
    area() float64
    perim() float64
}

// Implement the interface on rectangular and circle types
type rect struct {
    width, height float64
}
type circle struct {
    radius float64
}

// Per implementare un'interfaccia in Go basta implementare tutti i suoi
// metodi: nessuna dichiarazione "implements".

```

```go
// Implement area and perim methods on rect
func (r rect) area() float64 {
    return r.width * r.height
}
func (r rect) perim() float64 {
    return 2*r.width + 2*r.height
}

// Implement area and perim methods on circle
func (c circle) area() float64 {
    return math.Pi * c.radius * c.radius
}
func (c circle) perim() float64 {
    return 2 * math.Pi * c.radius
}

// Se una variabile ha tipo interfaccia si possono chiamare i metodi
// dell'interfaccia: measure funziona con qualsiasi tipo che implementi
// geometry.
func measure(g geometry) {
    fmt.Println(g)
    fmt.Println(g.area())
    fmt.Println(g.perim())
}

func main() {
    r := rect{width: 3, height: 4}
    c := circle{radius: 5}

    // rect e circle soddisfano entrambe geometry, quindi possono essere
    // passate a measure.
    measure(r)
    measure(c)
}
```

```
{3 4}
12
14
{5}
78.53981633974483
31.41592653589793
```

7.2.3 Analisi del codice

- `geometry` dichiara due metodi: `area()` e `perim()`.
- `rect` e `circle` sono struct che definiscono entrambi i metodi con i giusti receiver: *nient'altro* le collega a `geometry`.
- `measure(g geometry)` accetta qualunque tipo che implementi `geometry`; dentro la funzione si possono chiamare solo i metodi dell'interfaccia. La chiamata `g.area()` è risolta a runtime sul tipo concreto contenuto in `g`.
- `fmt.Println(g)` stampa il valore concreto: `{3 4}` per `rect` e `{5}` per `circle`.

- Output per `r` (3×4): area 12, perimetro 14; per `c` (raggio 5): area $25\pi \approx 78.5398$, perimetro $10\pi \approx 31.4159$.

Un secondo esempio mostra il polimorfismo in una collezione eterogenea:

Listato 7.4: `05_methods_interfaces/interface_slice.go`

```go
// interface_slice.go - Polimorfismo: una collezione eterogenea di forme.
//
// Esempio aggiuntivo (non presente nelle slide) che riusa solo concetti già
// visti: interfacce implicite, metodi e range su slice.
package main

import (
    "fmt"
    "math"
)

type shape interface {
    area() float64
}

type square struct{ side float64 }
type circle struct{ radius float64 }

func (s square) area() float64 { return s.side * s.side }
func (c circle) area() float64 { return math.Pi * c.radius * c.radius }

// totalArea somma le aree di tipi diversi: a runtime ogni chiamata
// s.area() viene risolta sul tipo concreto contenuto nell'interfaccia.
func totalArea(shapes []shape) float64 {
    total := 0.0
    for _, s := range shapes {
        total += s.area()
    }
    return total
}

func main() {
    shapes := []shape{square{2}, circle{1}, square{3}}
    fmt.Printf("area totale = %.4f\n", totalArea(shapes))
}
```

```
area totale = 16.1416
```

La somma è $4 + \pi + 9 \approx 16.1416$: `[]shape` contiene tipi diversi, e a ogni iterazione `s.area()` invoca l'implementazione giusta.

Errore comune: un metodo mancante

Se un tipo non implementa *tutti* i metodi, il compilatore rifiuta l'assegnazione all'interfaccia, spiegando quale metodo manca. Esempio con un tipo `T` che implementa solo `A()` assegnato a un'interfaccia che richiede `A()` e `B()`:

```
cannot use T{} (value of struct type T) as I value in variable declaration:
T does not implement I (missing method B)
```

Il controllo è fatto *a compile time*, quindi l'implementazione implicita non sacrifica la sicurezza dei tipi.

Confronto — C, Python, Java

In **Java** un tipo deve dichiarare esplicitamente `implements Interfaccia`, e quindi l'autore della classe deve conoscere l'interfaccia in anticipo. In **Go** la relazione è *strutturale*: si può definire un'interfaccia *dopo*, anche in un altro package, e i tipi esistenti la soddisfano se hanno i metodi giusti. In **Python** il *duck typing* ha un'idea simile («se cammina come un'anatra…»), ma il controllo avviene a runtime; in Go è statico. In **C** un'astrazione simile richiede tabelle di puntatori a funzione costruite a mano.

7.3 Gestione degli errori

7.3.1 Introduzione

Go usa **valori di errore** per indicare stati anomali, non eccezioni. Gli errori sono comunicati con un valore di ritorno *separato*. Come ha scritto R. Pike (citato nelle slide): «*Error handling [in Go] does not obscure the flow of control*», cioè la gestione degli errori non nasconde il flusso di controllo.

7.3.2 Funzionamento

- L'errore è, per **convenzione**, l'*ultimo* valore restituito.
- Un errore `nil` significa che **non si è verificato nessun errore**.
- Il chiamante controlla subito:

```
result, err := SomeFunction()
if err != nil {
    // gestisci l'errore
}
```

Il tipo `error` è un'**interfaccia predefinita**:

```
type error interface {
    Error() string
}
```

Qualunque tipo con un metodo `Error() string` soddisfa `error`, per la regola dell'implementazione implicita appena vista. `errors.New` costruisce un errore base con un messaggio.

Listato 7.5: `05_methods_interfaces/errors.go`

```
1 // errors.go - Gestione degli errori e interfaccia error (slide 69).
2 package main
3
4 import (
5     "errors"
6     "fmt"
```

```
)

// L'interfaccia predefinita error è:
//
//  type error interface { Error() string }
//
// Qualunque tipo con un metodo Error() string la soddisfa.

// divide segnala la condizione anomala con un errore invece che con
// un'eccezione. L'errore è l'ultimo valore restituito.
func divide(a, b float64) (float64, error) {
    if b == 0 {
        // errors.New costruisce un errore base con un messaggio.
        return 0, errors.New("divisione per zero")
    }
    return a / b, nil // nil = nessun errore
}

// Un tipo errore personalizzato: porta con sé dati strutturati.
type rangeError struct {
    value, max int
}

func (e rangeError) Error() string {
    return fmt.Sprintf("valore %d oltre il massimo %d", e.value, e.max)
}

func check(v int) error {
    if v > 10 {
        return rangeError{v, 10}
    }
    return nil
}

func main() {
    result, err := divide(10, 4)
    if err != nil {
        fmt.Println("errore:", err)
        return
    }
    fmt.Println("risultato:", result)

    if _, err := divide(1, 0); err != nil {
        fmt.Println("errore:", err)
    }

    for _, v := range []int{5, 42} {
        if err := check(v); err != nil {
            fmt.Println("check fallito:", err)
        } else {
            fmt.Println("check ok:", v)
        }
    }
}
```

```
risultato: 2.5
errore: divisione per zero
```

```
check ok: 5
check fallito: valore 42 oltre il massimo 10
```

7.3.3 Analisi del codice

- `divide` restituisce `(float64, error)`: in caso di divisore nullo restituisce un errore creato con `errors.New`, altrimenti `nil`. Dividere 10 per 4 dà 2,5; dividere per zero produce l'errore, che il chiamante stampa.
- `rangeError` è un tipo *definito dal programmatore* che implementa `error` con il metodo `Error()`: così l'errore può trasportare *dati strutturati* (qui il valore e il massimo).
- Il pattern `if err := check(v); err != nil { … }` usa l'istruzione di inizializzazione dell'`if` (Sezione 5.2).

Errore comune: ignorare l'errore

Scrivere `data, _ := os.ReadFile(...)` scarta l'errore e prosegue con un valore probabilmente vuoto. Poiché il linguaggio non obbliga a gestirlo (non esistono `try`/`catch`), la disciplina del `if err != nil` subito dopo la chiamata è la regola d'oro del codice Go.

Confronto — C, Python, Java

In **C** gli errori si segnalano con codici di ritorno o con `errno`. In **Java** e **Python** con eccezioni (`throw`/`raise`, `try`/`catch`/`except`), che interrompono il flusso e risalgono lo stack finché qualcuno le cattura. Go torna ai valori di ritorno del C, ma con un tipo dedicato (`error`) e risultati multipli: il flusso di controllo resta visibile nel codice. Il costo è un po' di verbosità (`if err != nil` ripetuto); il beneficio è che ogni punto che può fallire è evidente.

Capitolo 8

Concorrenza

8.1 La concorrenza in Go

Go fornisce la **concorrenza come parte del linguaggio di base**, non come libreria aggiuntiva. Gli strumenti sono due: le **goroutine**, che eseguono codice concorrentemente, e i **channel**, con cui le goroutine comunicano. Insieme implementano il modello **CSP** (Capitolo 1) e permettono di realizzare diversi *pattern* di concorrenza.

Il principio guida è: invece di far condividere memoria a più flussi di esecuzione proteggendola con lock, si fa in modo che i flussi *si scambino valori attraverso channel*. Go fornisce comunque anche i lock tradizionali (`sync.Mutex`), trattati nella Sezione 8.8.

Concorrenza e determinismo

I programmi concorrenti di questo capitolo sono stati eseguiti più volte. Dove l'output *può cambiare* da un'esecuzione all'altra (ordine di stampa delle goroutine, scelta casuale di `select`, risultato di una data race) il testo lo dichiara esplicitamente e l'output mostrato è *uno dei possibili*. In tutti gli altri casi l'output è deterministico.

8.2 Goroutine

8.2.1 Introduzione

Definizione — goroutine

Una **goroutine** è un *thread leggero* gestito dal runtime di Go.

Si avvia anteponendo `go` a una chiamata di funzione:

```
go f(x, y, z)    // avvia una nuova goroutine che esegue f(x, y, z)
```

Gli argomenti sono valutati nella goroutine *chiamante*; l'esecuzione di `f` avviene nella nuova goroutine, e il chiamante prosegue senza attendere.

8.2.2 Funzionamento

- Le goroutine **condividono lo stesso spazio di indirizzi**: per accedere a memoria condivisa serve sincronizzazione.
- **Attenzione**: quando `main` ritorna, il programma termina *immediatamente*, senza attendere il completamento delle altre goroutine.

Goroutine contro thread

Le goroutine *non sono thread* del sistema operativo, ma astrazioni leggere costruite sopra di essi.

- Lo **scheduler di Go** gestisce le goroutine, assegnandole ai thread del sistema operativo: *un thread OS può eseguire molte goroutine.*
- Le goroutine sono molto più economiche da creare e distruggere dei thread OS e consumano molta meno memoria. Le slide indicano come ordine di grandezza circa 5× in tempo di creazione/distruzione e circa 500× in memoria; sono valori *riportati dalle slide*, non misurati in questa guida, e dipendono da piattaforma e configurazione.

Ordine di esecuzione e priorità

Le goroutine **non hanno garanzie sull'ordine di esecuzione** rispetto all'ordine in cui sono state dichiarate, hanno tutte la **stessa priorità** e **non si può controllare** l'ordine con cui vengono eseguite.

Controllare le prestazioni: `GOMAXPROCS`

Le slide indicano che si può impostare `GOMAXPROCS` per controllare quanti thread del sistema operativo Go usa *simultaneamente*, tipicamente il numero di core virtuali della CPU. Si veda l'Appendice B per una precisazione sul nome: `GOMAXPROCS` è una *variabile d'ambiente*, mentre `runtime.GOMAXPROCS(n)` è la *funzione* del package `runtime` che fa la stessa cosa da codice (con `n = 0` restituisce solo il valore corrente).

8.2.3 Esempio: `main` termina prima delle goroutine

Listato 8.1: `06_concurrency/goroutine_termination.go`

```go
// goroutine_termination.go - ERRORE COMUNE: main termina prima delle
// goroutine (slide 47).
//
// Quando main ritorna, il programma termina immediatamente senza attendere
// le altre goroutine. L'output di questo programma è NON deterministico:
// dopo la riga di main possono comparire da 0 a 3 righe "goroutine N"
// (anche prima della riga di main, in ordine qualsiasi). Nelle prove fatte
// sono comparse quantità diverse da un'esecuzione all'altra: non c'è
// alcuna garanzia.
package main

import "fmt"

func say(id int) {
    fmt.Println("goroutine", id)
}

func main() {
    for i := 1; i <= 3; i++ {
        go say(i) // avvia la goroutine e prosegue subito
    }
    fmt.Println("main termina senza attendere")
```

```
    // Nessuna sincronizzazione: il programma esce qui.
    // Soluzione: vedi goroutine_wait.go (sync.WaitGroup).
}
```

Questo programma ha un output **non deterministico**. In sei esecuzioni si sono osservati esiti diversi; tre di essi sono:

```
# esecuzione A
main termina senza attendere

# esecuzione B
goroutine 3
goroutine 1
goroutine 2
main termina senza attendere

# esecuzione C
goroutine 1
main termina senza attendere
goroutine 2
```

Perché avviene

L'istruzione `go say(i)` *non attende* che `say` venga eseguita. Dopo aver avviato le tre goroutine, `main` stampa il proprio messaggio e ritorna: a quel punto il processo termina, e le goroutine che non hanno ancora avuto il tempo di stampare vengono abbandonate. Quante ne riescano a stampare dipende da come lo scheduler le assegna ai thread: da zero a tre, in ordine qualsiasi. Il programma è quindi *scorretto*, anche se in alcune esecuzioni sembra funzionare.

8.2.4 La correzione: attendere con `sync.WaitGroup`

Il rimedio è sincronizzare `main` con le goroutine. Lo strumento è `sync.WaitGroup` (Sezione 8.9).

Listato 8.2: `06_concurrency/goroutine_wait.go`

```
// goroutine_wait.go - Versione corretta di goroutine_termination.go:
// main attende le goroutine con sync.WaitGroup (slide 47, 64).
//
// L'ORDINE con cui le goroutine stampano non è garantito: tutte hanno la
// stessa priorità e lo scheduler decide. Quindi l'output varia da
// un'esecuzione all'altra (ma contiene sempre le tre righe).
package main

import (
    "fmt"
    "sync"
)

func say(id int, wg *sync.WaitGroup) {
    defer wg.Done() // decrementa il contatore all'uscita, anche in caso di return
    ↪ anticipato
    fmt.Println("goroutine", id)
}

```

```
19 func main() {
20     var wg sync.WaitGroup
21     for i := 1; i <= 3; i++ {
22         wg.Add(1) // PRIMA di avviare la goroutine, non al suo interno
23         go say(i, &wg)
24     }
25     wg.Wait() // blocca finché il contatore torna a zero
26     fmt.Println("main: tutte le goroutine sono terminate")
27 }
```

Ogni esecuzione stampa sempre le tre righe delle goroutine *prima* della riga finale di `main`, ma l'ordine delle tre goroutine varia. Due esecuzioni:

```
goroutine 3
goroutine 1
goroutine 2
main: tutte le goroutine sono terminate

goroutine 2
goroutine 1
goroutine 3
main: tutte le goroutine sono terminate
```

Analisi del codice

- `wg.Add(1)` incrementa il contatore *prima* di avviare la goroutine. Se `Add` fosse chiamata *dentro* la goroutine, `wg.Wait()` potrebbe eseguirsi quando il contatore è ancora zero e ritornare subito.
- `defer wg.Done()` decrementa il contatore all'uscita di `say`, anche in caso di `return` anticipato.
- `wg.Wait()` blocca `main` finché il contatore torna a zero.
- Il `*sync.WaitGroup` è passato *per puntatore*: se fosse passato per valore, ogni goroutine decrementerebbe una *copia* e `Wait` non terminerebbe mai.

8.2.5 Molte goroutine, pochi thread

Il programma seguente avvia 10 000 goroutine, ben più dei thread OS disponibili. Lo scheduler le distribuisce sui thread. Ogni goroutine scrive in una posizione *diversa* della slice, quindi non ci sono accessi concorrenti allo stesso elemento. Il valore di `GOMAXPROCS` dipende dalla macchina: per rendere l'output identico ovunque si stampa solo un controllo di coerenza.

Listato 8.3: `06_concurrency/gomaxprocs.go`

```
1 // gomaxprocs.go - Goroutine leggere e thread del sistema operativo
2 // (slide 47-48).
3 //
4 // runtime.GOMAXPROCS(0) restituisce il numero massimo di thread OS che
5 // possono eseguire codice Go contemporaneamente, senza modificarlo. Il
6 // valore dipende dalla macchina, quindi viene stampato solo un controllo
7 // di coerenza per mantenere l'output identico ovunque.
8 //
9 // Qui si avviano 10000 goroutine, molte più dei thread disponibili: lo
```

```go
// scheduler di Go le multiplexa sui thread OS. Con 10000 thread OS reali
// il costo in memoria e tempo di creazione sarebbe molto maggiore.
package main

import (
    "fmt"
    "runtime"
    "sync"
)

func main() {
    fmt.Println("GOMAXPROCS(0) >= 1:", runtime.GOMAXPROCS(0) >= 1)

    const n = 10000
    finished := make([]bool, n) // una posizione per goroutine: nessuna race
    var wg sync.WaitGroup
    for i := 0; i < n; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            finished[i] = true
        }()
    }
    wg.Wait()

    count := 0
    for _, f := range finished {
        if f {
            count++
        }
    }
    fmt.Println("goroutine completate:", count)
}
```

```
GOMAXPROCS(0) >= 1: true
goroutine completate: 10000
```

Confronto — C, Python, Java

Un thread POSIX (`pthread`) o un thread Java *tradizionale* corrisponde a un thread del kernel, con uno stack di dimensione tipicamente dell'ordine del megabyte: crearne decine di migliaia è costoso. Una goroutine parte con uno stack molto piccolo e cresciuto dinamicamente, ed è schedulata dal *runtime* in spazio utente, quindi se ne possono avere centinaia di migliaia. In **Python** (CPython) i thread sono thread del sistema operativo, ma il *Global Interpreter Lock* impedisce l'esecuzione *parallela* di bytecode Python in più thread; le goroutine invece possono girare in parallelo su più core. In Go non c'è un oggetto `Thread` da creare e gestire: si scrive `go f()`.

8.3 Channel

8.3.1 Introduzione

I **channel** permettono una comunicazione *efficiente e sicura* tra goroutine tramite l'invio di valori. Un channel è una **coda thread-safe** gestita dal runtime di Go; nasconde la complessità della comunicazione tra thread (le slide precisano che internamente usa meccanismi di sincronizzazione come mutex e semafori). Più goroutine possono inviare valori nello stesso channel: è ideale per notifiche, multiplexing, … e la comunicazione è thread-safe *«out of the box»*.

Definizione — channel

Un **channel** è un *condotto tipizzato* attraverso il quale una goroutine può inviare e ricevere valori con l'operatore `<-`. I dati fluiscono *nella direzione della freccia*.

8.3.2 Sintassi

```
ch <- v         // invia v sul channel ch
v := <-ch       // riceve da ch e assegna a v
ch := make(chan int)   // crea un channel di int
```

- Un channel trasporta valori di un tipo specifico (`int`, struct, …). Per impostazione predefinita è **bidirezionale**.
- Va creato con `make` prima dell'uso. Il valore zero di un channel è `nil`.
- **Invio e ricezione bloccano** finché l'altro lato non è pronto (per i channel senza buffer, vedi Sezione 8.4). Le goroutine possono quindi **sincronizzarsi** senza lock espliciti né variabili di condizione.

8.3.3 Un channel come sincronizzazione

Il programma seguente (esempio aggiuntivo) mostra l'uso più semplice di un channel come meccanismo di sincronizzazione: `main` attende la fine di una goroutine ricevendo da un channel `done`.

Listato 8.4: `06_concurrency/channel_sync.go`

```
1  // channel_sync.go - Un channel come meccanismo di sincronizzazione
2  // (slide 50).
3  //
4  // Esempio aggiuntivo: main attende la fine di una goroutine ricevendo da un
5  // channel "done". Non servono lock né condition variable.
6  package main
7
8  import (
9      "fmt"
10     "time"
11 )
12
13 func worker(done chan bool) {
14     fmt.Println("worker: inizio lavoro...")
```

```
    time.Sleep(100 * time.Millisecond) // lavoro simulato
    fmt.Println("worker: fatto")
    done <- true // segnala il completamento
}

func main() {
    done := make(chan bool)
    go worker(done)

    // La ricezione blocca main finché il worker non invia.
    <-done
    fmt.Println("main: il worker ha terminato")
}
```

```
worker: inizio lavoro...
worker: fatto
main: il worker ha terminato
```

La ricezione `<-done` blocca `main` finché il worker non invia `true`; quindi «main: il worker ha terminato» compare sempre dopo «worker: fatto».

8.3.4 Esempio: somma distribuita

La somma di una slice è distribuita tra due goroutine: è il pattern **SPMD** (*Single Program Multiple Data*), in cui ogni goroutine esegue lo stesso programma su una porzione diversa dei dati.

Listato 8.5: `06_concurrency/channel_sum.go`

```
// channel_sum.go - Somma distribuita tra due goroutine (slide 51).
//
// Pattern SPMD (Single Program Multiple Data): ogni goroutine esegue lo
// stesso codice su una porzione diversa dei dati.
package main

import "fmt"

func sum(s []int, c chan int) {
    sum := 0
    for _, v := range s {
        sum += v
    }
    c <- sum // send sum to c
}

func main() {
    s := []int{7, 2, 8, -9, 4, 0}
    c := make(chan int)
    go sum(s[:len(s)/2], c) // prima metà: 7+2+8 = 17
    go sum(s[len(s)/2:], c) // seconda metà: -9+4+0 = -5

    // Le due ricezioni bloccano finché le goroutine non inviano.
    // Non si sa quale somma arrivi per prima, ma il totale è lo stesso
    // (l'addizione è commutativa).
    sum1, sum2 := <-c, <-c
```

```
    fmt.Println("Total sum:", sum1+sum2)
}
```

```
Total sum: 12
```

Analisi del codice e flusso di esecuzione

1. `main` crea il channel `c` (senza buffer) e lancia due goroutine `sum`, sulla prima e sulla seconda metà di `s`: $7+2+8=17$ e $-9+4+0=-5$.
2. `sum1, sum2 := <-c, <-c` esegue due ricezioni. Ciascuna *blocca* `main` finché una goroutine non invia il proprio risultato.
3. Non si sa quale somma arrivi per prima, ma il totale $17+(-5)=12$ è lo stesso perché l'addizione è commutativa: l'output è deterministico anche se l'ordine di arrivo non lo è.

8.3.5 Esempio: generatore di Fibonacci

Listato 8.6: `06_concurrency/channel_fib.go`

```go
// channel_fib.go - Generatore di Fibonacci con goroutine e channel
// (slide 52).
package main

import "fmt"

// fib è un produttore: invia numeri di Fibonacci sul channel c, per
// sempre. Ogni invio blocca finché il main non riceve (channel senza
// buffer), quindi il produttore non va "avanti" rispetto al consumatore.
func fib(c chan int) {
    x, y := 1, 1
    for {
        c <- x
        x, y = y, x+y
    }
}

func main() {
    c := make(chan int)
    go fib(c)
    for i := 0; i < 10; i++ {
        fmt.Println(<-c)
    }
    // Quando main termina, la goroutine fib (bloccata su un invio)
    // viene semplicemente abbandonata: si veda select per una chiusura
    // ordinata.
}
```

```
1
1
2
3
5
8
```

```
13
21
34
55
```

`fib` è un *produttore* che invia per sempre sul channel; `main` è il *consumatore* e riceve solo 10 valori. Poiché il channel non ha buffer, ogni invio blocca finché `main` non riceve: il produttore non può «correre avanti». Dopo la decima ricezione, `main` termina e la goroutine `fib` (bloccata su un invio) viene semplicemente abbandonata: è accettabile qui perché il programma finisce, ma in un programma di lunga durata una goroutine bloccata per sempre è una *perdita* (*goroutine leak*). La versione con `select` e un channel `quit` (Sezione 8.6) la ferma in modo ordinato.

Confronto — C, Python, Java

Un channel somiglia a una coda bloccante: in **Java** `SynchronousQueue` (senza buffer) o `ArrayBlockingQueue` (con buffer), in **Python** `queue.Queue`, in **C** un *pipe* o una coda protetta da mutex e condition variable scritta a mano. Le differenze caratteristiche di Go sono che il channel è un tipo *di primo livello* del linguaggio, ha tipo statico, direzione e operazioni di `close`, e si combina con `select`.

8.4 Channel con e senza buffer

Channel senza buffer (*unbuffered*). Per impostazione predefinita le operazioni su un channel bloccano. Secondo la specifica del linguaggio: se la capacità è zero o assente, il channel è senza buffer e la comunicazione riesce *solo quando mittente e ricevente sono entrambi pronti*; in un channel senza buffer il mittente resta bloccato finché il ricevente non ha ricevuto il valore.

Channel con buffer (*buffered*). Non bloccano se non sono pieni (per l'invio) o non sono vuoti (per la ricezione). Si specifica la capacità alla creazione: `ch := make(chan int, 100)`. L'invio blocca *solo* se il buffer è pieno; la ricezione blocca *solo* se il buffer è vuoto. È utile per *disaccoppiare* le goroutine: possono procedere senza sincronizzarsi a ogni passo.

Listato 8.7: `06_concurrency/channel_buffered.go`

```go
// channel_buffered.go - Channel con e senza buffer (slide 53).
package main

import "fmt"

func main() {
    // Con buffer di capacità 2: i primi due invii NON bloccano anche se
    // nessuno sta ancora ricevendo.
    ch := make(chan int, 2)
    ch <- 1
    ch <- 2
    fmt.Println("len =", len(ch), "cap =", cap(ch)) // len=2 cap=2

    // Un terzo invio bloccherebbe (buffer pieno) e, non esistendo altre
    // goroutine che ricevano, causerebbe un deadlock. Le ricezioni
    // seguono l'ordine FIFO.
    fmt.Println(<-ch) // 1
    fmt.Println(<-ch) // 2
```

```
19
20      // Un channel senza buffer (capacità 0) richiede sender e receiver
21      // pronti contemporaneamente: l'invio avviene in una goroutine perché
22      // altrimenti main si bloccherebbe da solo.
23      unbuf := make(chan string)
24      go func() { unbuf <- "ping" }()
25      fmt.Println(<-unbuf)
26  }
```

```
len = 2 cap = 2
1
2
ping
```

8.4.1 Analisi del codice

- Con `make(chan int, 2)` i primi due invii non bloccano anche se nessuno riceve; `len` e `cap` dicono quanti elementi ci sono e quanti ne entrano. Le ricezioni seguono l'ordine **FIFO**.
- Un *terzo* invio senza ricezioni nel frattempo bloccherebbe per sempre: nel `main` da solo, il runtime lo rileverebbe come deadlock (Sezione 8.11).
- Nel channel senza buffer, l'invio è fatto in una goroutine apposita: eseguito direttamente da `main` bloccherebbe `main` stesso, prima che arrivi la ricezione sulla riga successiva.

Una precisazione sulle slide

Una slide afferma che «invio e ricezione bloccano finché l'altro lato è pronto»; questa descrizione è esatta per i channel *senza buffer*. Con un buffer (slide successive) i blocchi si verificano solo a buffer pieno (invio) o vuoto (ricezione). Le due affermazioni sono coerenti se lette in quest'ordine.

8.5 Chiudere un channel e iterare con `range`

8.5.1 `close`

Un channel si chiude con `close(ch)`. Le regole sono:

- il/i **mittente/i** devono smettere di inviare dopo la chiusura;
- il/i **ricevente/i** possono ancora ricevere i *valori già nel buffer*;
- i riceventi possono verificare se il channel è chiuso con la forma a due valori `v, ok := <-ch`: `ok` è `false` se il channel è chiuso *e* non ci sono più valori da ricevere;
- **solo il mittente** (non il ricevente) dovrebbe chiudere il channel;
- **inviare su un channel chiuso** causa un **panic**: `panic: send on closed channel`;
- ricevere da un channel chiuso e vuoto restituisce subito il *valore zero* (con `ok == false`).

Listato 8.8: `06_concurrency/channel_close.go`

```go
// channel_close.go - close e rilevamento della chiusura (slide 54).
package main

import "fmt"

func main() {
    jobs := make(chan int, 5)
    done := make(chan bool)

    // Il consumatore legge finché il channel è chiuso E vuoto.
    go func() {
        for {
            j, more := <-jobs // more == false: chiuso e senza altri valori
            if more {
                fmt.Println("received job", j)
            } else {
                fmt.Println("received all jobs")
                done <- true
                return
            }
        }
    }()

    // Il produttore (il sender) invia e poi chiude: solo il sender
    // dovrebbe chiudere un channel.
    for j := 1; j <= 3; j++ {
        jobs <- j
        fmt.Println("sent job", j)
    }
    close(jobs)
    fmt.Println("sent all jobs")

    <-done // attende il consumatore

    // Ricevere da un channel chiuso e vuoto restituisce subito il valore
    // zero e ok == false.
    v, ok := <-jobs
    fmt.Println(v, ok) // 0 false
}
```

```
sent job 1
sent job 2
sent job 3
sent all jobs
received job 1
received job 2
received job 3
received all jobs
0 false
```

Analisi del codice

Il consumatore (goroutine anonima) legge in un ciclo con `j, more := <-jobs`: finché `more` è vero stampa il job, appena diventa falso (channel chiuso *e* vuoto) segnala la fine su `done`. Il

produttore (`main`) invia tre job, chiude il channel, e attende `done`. In fondo la ricezione da un channel già chiuso e vuoto dà `0 false`.

Osservazione

Nell'output l'*intercalarsi* delle righe «sent» e «received» non è garantito: dipende dallo scheduler. Qui il buffer di 5 posti permette a `main` di inviare tutti e tre i job prima che il consumatore parta, ma in un'altra esecuzione le righe potrebbero alternarsi. Ciò che è garantito è l'ordine interno (job 1, 2, 3) e che «received all jobs» segua tutti i «received job».

Errore comune: inviare su un channel chiuso

L'invio su un channel chiuso non restituisce un errore ma provoca un *panic*: si arresta l'intero programma. Per questo la chiusura spetta al mittente, che sa quando non ha più nulla da inviare; un ricevente non ha modo di sapere se altri mittenti stanno ancora inviando. L'esempio seguente termina *volutamente* con un panic.

Listato 8.9: `06_concurrency/pitfall_send_closed.go`

```go
// pitfall_send_closed.go - ERRORE COMUNE: invio su un channel chiuso
// (slide 54).
//
// ATTENZIONE: questo programma TERMINA VOLUTAMENTE con un panic:
//
//  panic: send on closed channel
//
// Si noti che la RICEZIONE da un channel chiuso è invece lecita.
package main

import "fmt"

func main() {
    ch := make(chan int, 2)
    ch <- 1
    close(ch)

    v, ok := <-ch // ricezione dal buffer residuo: 1 true
    fmt.Println(v, ok)
    v, ok = <-ch // channel chiuso e vuoto: 0 false
    fmt.Println(v, ok)

    fmt.Println("invio su channel chiuso...")
    ch <- 2 // PANIC
}
```

```
1 true
0 false
invio su channel chiuso...
panic: send on closed channel

goroutine 1 [running]:
main.main()
        .../pitfall_send_closed.go:24 +0x1a5
exit status 2
```

La ricezione dopo `close` è lecita (prima il valore nel buffer, `1 true`, poi il valore zero, `0 false`); l'invio no.

8.5.2 `range` su un channel

Con `range` si ricevono ripetutamente valori da un channel *finché non viene chiuso*:

```
for elem := range ch {
    fmt.Println(elem)
}
```

Listato 8.10: `06_concurrency/channel_range.go`

```
// channel_range.go - range su un channel (slide 55).
package main

import "fmt"

func main() {
    ch := make(chan int, 3)

    go func() {
        for i := 1; i <= 5; i++ {
            ch <- i
        }
        // Senza close, il range sotto non terminerebbe mai e main
        // andrebbe in deadlock dopo aver letto i cinque valori.
        close(ch)
    }()

    // Riceve ripetutamente finché il channel è chiuso e svuotato.
    for elem := range ch {
        fmt.Println(elem)
    }
    fmt.Println("channel chiuso, ciclo terminato")
}
```

```
1
2
3
4
5
channel chiuso, ciclo terminato
```

Errore comune: dimenticare `close` con `range`

Il ciclo `for elem := range ch` termina *solo* quando `ch` è chiuso e svuotato. Se il mittente dimentica `close(ch)`, dopo l'ultimo valore il ciclo si blocca in attesa di altri: se non esistono altre goroutine in grado di inviare, il runtime segnala un deadlock.

8.6 `select`

8.6.1 Introduzione e sintassi

`select` permette a una goroutine di **attendere su più channel contemporaneamente**. Blocca finché uno dei suoi `case` può procedere, poi esegue quel caso.

- Se *più* case sono pronti, ne viene scelto **uno a caso**.
- Se nessun case è pronto e c'è un ramo `default`, viene eseguito quello (comportamento non bloccante).

```
select {
case msg1 := <-ch1:    // ricezione su ch1
    // gestisci msg1
case msg2 := <-ch2:    // ricezione su ch2
    // gestisci msg2
}
```

È utile per il *multiplexing* e per gestire i *timeout*.

Un refuso nelle slide

Nel frammento della slide la variabile della prima ricezione è scritta `mgs1` (e poi commentata come `msg1`). È un errore di battitura: il nome corretto, usato nei listati di questa guida, è `msg1`.

8.6.2 Esempio: Fibonacci con `select` e un channel `quit`

Versione del generatore con due channel: `ch` per inviare i valori e `quit` per segnalare quando fermarsi.

Listato 8.11: `06_concurrency/channel_fib_select.go`

```
1  // channel_fib_select.go - Fibonacci con select e channel quit (slide 57-58).
2  package main
3
4  import "fmt"
5
6  func fibonacci(ch, quit chan int) {
7      x, y := 1, 1
8      for {
9          select {
10         case ch <- x: // invia il numero di Fibonacci corrente
11             x, y = y, x+y
12         case <-quit: // riceve il segnale di terminazione
13             fmt.Println("quit")
14             return
15         }
16     }
17 }
18
19 func main() {
20     ch := make(chan int) // channel senza buffer
```

```
    quit := make(chan int)

    go func() { // goroutine anonima: il consumatore
        for i := 0; i < 10; i++ {
            fmt.Println(<-ch) // riceve un valore di Fibonacci
        }
        quit <- 0 // dice a fibonacci di fermarsi
    }()

    fibonacci(ch, quit) // eseguita nella goroutine principale
}
```

```
1
1
2
3
5
8
13
21
34
55
quit
```

Analisi del codice e flusso di esecuzione

1. `main` avvia una goroutine anonima (il consumatore) e poi esegue `fibonacci(ch, quit)` *nella goroutine principale*.
2. Dentro `fibonacci`, il `select` offre due possibilità: inviare `x` su `ch` (pronto solo quando il consumatore sta ricevendo) o ricevere da `quit` (pronto solo quando qualcuno ha inviato).
3. Il consumatore riceve 10 valori, poi invia su `quit`. Da quel momento il ramo `quit` del `select` diventa pronto mentre il ramo di invio non lo è più (nessuno riceve): `fibonacci` stampa «quit» e ritorna, terminando in modo *pulito*.

Errore nella slide: variabile non definita

Nelle slide `main` dichiara `ch := make(chan int)` ma poi chiama `fibonacci(c, quit)`: la variabile `c` *non esiste* e il programma non compila (errore reale: `undefined: c`). Si tratta di un **errore di trascrizione** nella slide, non una scelta tecnica: il listato di questa guida usa il nome corretto `ch`. Si veda anche l'Appendice B.

8.6.3 Scelta casuale tra case pronti

Se entrambi i case sono pronti, la scelta è casuale. Il programma seguente ha due channel già pieni:

Listato 8.12: `06_concurrency/select_multiplex.go`

```
// select_multiplex.go - select su più channel e scelta casuale (slide 56).
package main

import "fmt"
```

```
5
6  func main() {
7      ch1 := make(chan string, 1)
8      ch2 := make(chan string, 1)
9      ch1 <- "da ch1"
10     ch2 <- "da ch2"
11
12     // Entrambi i case sono pronti: select ne sceglie UNO a caso. L'ordine
13     // delle due righe stampate può quindi variare da esecuzione a
14     // esecuzione, ma entrambi i messaggi vengono sempre ricevuti.
15     for i := 0; i < 2; i++ {
16         select {
17         case msg1 := <-ch1:
18             fmt.Println("ricevuto", msg1)
19         case msg2 := <-ch2:
20             fmt.Println("ricevuto", msg2)
21         }
22     }
23 }
```

L'output è **non deterministico**: l'ordine delle due righe varia. In sei esecuzioni consecutive si sono osservate entrambe le sequenze:

```
ricevuto da ch1
ricevuto da ch2

ricevuto da ch2
ricevuto da ch1
```

La scelta casuale evita che un channel «affami» gli altri.

8.6.4 `select` non bloccante con `default`

Con un ramo `default` il `select` *non blocca* se i channel non sono pronti. I case possono essere anche invii:

Listato 8.13: `06_concurrency/select_default.go`

```
1  // select_default.go - select non bloccante con default (slide 56, 59).
2  package main
3
4  import "fmt"
5
6  func main() {
7      ch1 := make(chan string)
8      ch3 := make(chan string, 1)
9
10     // Ricezione non bloccante: nessun valore pronto -> scatta default.
11     select {
12     case msg := <-ch1:
13         fmt.Println("ricevuto", msg)
14     default:
15         fmt.Println("nessun messaggio pronto")
16     }
17
18     // Invio non bloccante: ch3 ha un posto libero nel buffer.
```

```
    select {
    case ch3 <- "ciao":
        fmt.Println("inviato su ch3")
    default:
        fmt.Println("ch3 pieno")
    }

    // Secondo invio: il buffer è ora pieno, quindi si va in default.
    select {
    case ch3 <- "altro":
        fmt.Println("inviato su ch3")
    default:
        fmt.Println("ch3 pieno")
    }

    // Ora c'è un valore pronto su ch3: viene scelto il case di ricezione.
    select {
    case msg := <-ch3:
        fmt.Println("ricevuto", msg)
    default:
        fmt.Println("nessun messaggio pronto")
    }
}
```

```
nessun messaggio pronto
inviato su ch3
ch3 pieno
ricevuto ciao
```

Analisi del codice

Nel primo `select` nessuno ha inviato su `ch1`: scatta `default`. `ch3` ha buffer 1: il primo invio riesce (spazio libero), il secondo trova il buffer pieno e va in `default`; infine la ricezione trova un valore pronto. Senza `default`, ognuna di queste operazioni bloccherebbe.

8.7 Timer

Un **timer** fornisce un channel (il campo `C`) su cui riceve un valore quando la durata specificata è trascorsa. Si usa per attendere un tempo preciso:

```
timer := time.NewTimer(time.Second * 2)   // attende 2 secondi
<-timer.C                                  // blocca finché il timer scatta
```

Un timer può essere **annullato** prima che scatti con `Stop()`. È utile per implementare:

- **timeout**: attendere un evento o il completamento di un'operazione prima di procedere;
- **ritardi**: attendere un periodo prima di eseguire un compito.

Listato 8.14: `06_concurrency/timers.go`

```
// timers.go - Timer e timeout con select (slide 56, 60).
```

```go
package main

import (
    "fmt"
    "time"
)

func main() {
    // Attesa di una durata: il timer espone un channel C su cui riceve
    // un valore quando scade.
    start := time.Now()
    timer := time.NewTimer(200 * time.Millisecond)
    <-timer.C // blocca fino allo scadere
    fmt.Println("timer 1 scaduto, trascorsi almeno 200ms:",
        time.Since(start) >= 200*time.Millisecond)

    // Un timer si può annullare con Stop() prima che scatti.
    timer2 := time.NewTimer(time.Second)
    stopped := timer2.Stop()
    fmt.Println("timer 2 annullato:", stopped)

    // Timeout: select attende un risultato oppure il timer, a seconda di
    // quale arrivi prima.
    result := make(chan string)
    go func() {
        time.Sleep(500 * time.Millisecond) // operazione lenta
        result <- "risultato"
    }()

    // Anche il timeout è un timer: il suo channel C si "attiva" allo
    // scadere dei 100ms. Vince il case che diventa pronto per primo.
    timeout := time.NewTimer(100 * time.Millisecond)
    select {
    case r := <-result:
        fmt.Println(r)
        timeout.Stop()
    case <-timeout.C:
        fmt.Println("timeout: l'operazione è troppo lenta")
    }
}
```

```
timer 1 scaduto, trascorsi almeno 200ms: true
timer 2 annullato: true
timeout: l'operazione è troppo lenta
```

Analisi del codice

Il primo timer blocca `main` per 200 ms; il programma verifica solo che sia trascorso *almeno* quel tempo (la durata esatta non è deterministica, ma non può essere inferiore). `Stop()` annulla il secondo timer: restituisce `true` perché il timer non era ancora scattato. L'ultimo blocco realizza un *timeout* con `select`: l'operazione lenta impiega 500 ms, il timer da 100 ms scatta per primo e vince il relativo `case`. Quella goroutine resterà in attesa di inviare su un channel senza ricevente, ma qui il programma termina subito.

8.8 Mutua esclusione

8.8.1 Esercizio: un mutex realizzato con un channel

Go fornisce i *mutex* per garantire l'accesso sicuro ai dati condivisi tra goroutine; `sync.Mutex` è lo strumento usuale per bloccare e sbloccare. Le slide propongono anche un esercizio: realizzare un mutex *usando un channel*, completando lo scheletro

```
type Lock struct { /* ? */ }
func NewLock() Lock       { /* ? */ }
func (l *Lock) Lock()     { /* ? */ }
func (l *Lock) Unlock()   { /* ? */ }
```

Idea della soluzione. Un channel con buffer di capacità 1 contiene al più un «gettone». Chi possiede il gettone detiene il lock; chi non lo trova si blocca sulla ricezione:

- `NewLock` crea il channel con buffer 1 e vi inserisce un valore: il lock parte *libero* (gettone presente);
- `Lock()` *riceve* dal channel: se il gettone c'è lo prende (il lock è ora occupato), altrimenti la goroutine si blocca finché qualcuno lo restituisce;
- `Unlock()` *invia* sul channel: restituisce il gettone, liberando il lock e svegliando eventualmente una goroutine in attesa.

L'invariante è: gettoni nel channel = 1 se il lock è libero, 0 se occupato.

Listato 8.15: `06_concurrency/mutex_using_channel.go`

```
1  // mutex_using_channel.go - Esercizio: un mutex realizzato con un channel
2  // (slide 61-62).
3  //
4  // Idea: un channel con buffer di capacità 1 contiene al più un "gettone".
5  // Chi possiede il gettone detiene il lock; chi lo trova assente si blocca
6  // sulla ricezione.
7  package main
8
9  import (
10     "fmt"
11     "sync"
12 )
13
14 // Nota: "go vet" segnala "return copies lock value" su NewLock perché il
15 // tipo ha metodi Lock/Unlock e vet applica l'euristica dei lock non
16 // copiabili (come sync.Mutex). Qui la copia è innocua: la struct contiene
17 // solo un riferimento al channel, quindi tutte le copie condividono lo
18 // stesso "gettone". Il codice è quello della slide.
19 type Lock struct {
20     ch chan bool
21 }
22
23 func NewLock() Lock {
24     lock := Lock{make(chan bool, 1)} // channel con buffer
25     lock.ch <- true                  // gettone iniziale: il lock parte libero
26     return lock
27 }
```

```
28
29 // Lock prende il gettone; se non c'è, blocca finché qualcuno lo restituisce.
30 func (l *Lock) Lock() {
31     <-l.ch // receive
32 }
33
34 // Unlock restituisce il gettone.
35 func (l *Lock) Unlock() {
36     l.ch <- true // send
37 }
38
39 func main() {
40     lock := NewLock()
41     counter := 0
42
43     var wg sync.WaitGroup
44     for i := 0; i < 100; i++ {
45         wg.Add(1)
46         go func() {
47             defer wg.Done()
48             for j := 0; j < 1000; j++ {
49                 lock.Lock()
50                 counter++ // sezione critica
51                 lock.Unlock()
52             }
53         }()
54     }
55     wg.Wait()
56     fmt.Println("counter =", counter) // sempre 100000
57 }
```

```
counter = 100000
```

Cento goroutine incrementano mille volte la stessa variabile `counter` proteggendola con il lock: il risultato è sempre $100 \times 1000 = 100000$.

`go vet` e la copia del lock

Il codice è quello della slide. L'analizzatore `go vet` segnala `return copies lock value` su `NewLock`, perché il tipo ha metodi `Lock`/`Unlock` e vet applica l'euristica dei «lock non copiabili» (come `sync.Mutex`, che *non* va copiato). Nel nostro caso la copia è innocua: la struct contiene solo il *riferimento* a un channel, e tutte le copie condividono lo stesso gettone. Le slide stesse notano che, nella pratica, `sync.Mutex` è più idiomatico ed efficiente.

8.8.2 `sync.Mutex`

Mutex (`sync.Mutex`): garantisce la *mutua esclusione* e previene le *race condition*; solo una goroutine alla volta può accedere alla *sezione critica*. L'uso è:

```
mu.Lock()
counter++
mu.Unlock()
```

Listato 8.16: 06_concurrency/mutex.go

```go
// mutex.go - sync.Mutex per la mutua esclusione (slide 63).
package main

import (
    "fmt"
    "sync"
)

// Counter protegge il campo n con un mutex.
type Counter struct {
    mu sync.Mutex
    n  int
}

func (c *Counter) Inc() {
    c.mu.Lock()
    // defer garantisce lo Unlock anche in caso di return anticipato o panic.
    defer c.mu.Unlock()
    c.n++
}

func main() {
    var c Counter
    var wg sync.WaitGroup

    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for j := 0; j < 1000; j++ {
                c.Inc()
            }
        }()
    }
    wg.Wait()
    fmt.Println("counter =", c.n) // sempre 100000
}
```

```
counter = 100000
```

Si noti `defer c.mu.Unlock()` subito dopo il `Lock`: garantisce il rilascio anche in caso di `return` anticipato o di panic, ed è l'applicazione tipica di `defer` (Sezione 5.4).

Errore comune: mancata sincronizzazione (data race)

Se si omette il mutex, l'operazione `counter++` *non è atomica*: si compone di lettura, incremento e scrittura. Due goroutine possono leggere lo stesso valore, incrementarlo e riscrivere *lo stesso* risultato, perdendo un aggiornamento. L'esempio seguente ha un output **non deterministico** (di solito inferiore a 100000).

Listato 8.17: 06_concurrency/race_counter.go

```go
// race_counter.go - ERRORE COMUNE: data race su una variabile condivisa
// (slide 47, 63).
```

```go
//
// counter++ NON è atomico (legge, incrementa, scrive). Con più goroutine
// non sincronizzate alcuni aggiornamenti si perdono. Il risultato è
// NON deterministico e quasi sempre inferiore a 100000. (In rari casi su
// macchine con un solo thread potrebbe risultare esatto.)
package main

import (
    "fmt"
    "sync"
)

func main() {
    counter := 0
    var wg sync.WaitGroup

    for i := 0; i < 100; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for j := 0; j < 1000; j++ {
                counter++ // DATA RACE: nessuna mutua esclusione
            }
        }()
    }
    wg.Wait()
    fmt.Println("atteso 100000, ottenuto", counter)
    // Correzione: vedi mutex.go (sync.Mutex).
}
```

```
atteso 100000, ottenuto 94808
atteso 100000, ottenuto 99120
atteso 100000, ottenuto 95983
```

(Tre esecuzioni diverse: i valori cambiano a ogni lancio.) Il bug è insidioso perché il programma compila, termina e produce un numero plausibile: l'errore è nella *perdita* silenziosa di aggiornamenti. Si risolve con `sync.Mutex` (o con un channel come sopra).

Un mutex non è rientrante

Se una goroutine chiama `Lock()` mentre già detiene lo stesso mutex, si blocca per sempre su se stessa (deadlock): a differenza dei `synchronized`/`ReentrantLock` di Java, `sync.Mutex` non è rientrante.

8.8.3 `sync.RWMutex`

Il **mutex lettori/scrittore** (`sync.RWMutex`) consente *più lettori oppure un solo scrittore*. Migliora le prestazioni quando le letture sono frequenti e le scritture rare.

- **Lettori**: `RLock()` / `RUnlock()` → accesso *condiviso*;
- **Scrittore**: `Lock()` / `Unlock()` → accesso *esclusivo*.

Listato 8.18: 06_concurrency/rwmutex.go

```go
// rwmutex.go - sync.RWMutex: molti lettori oppure un solo scrittore
// (slide 63).
package main

import (
    "fmt"
    "sync"
)

// Store è una cache chiave-valore letta spesso e scritta di rado.
type Store struct {
    mu   sync.RWMutex
    data map[string]int
}

// Get usa un lock di LETTURA: più Get possono procedere insieme.
func (s *Store) Get(k string) (int, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    v, ok := s.data[k]
    return v, ok
}

// Set usa il lock di SCRITTURA: accesso esclusivo (blocca lettori e altri
// scrittori).
func (s *Store) Set(k string, v int) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.data[k] = v
}

func main() {
    s := &Store{data: make(map[string]int)}
    s.Set("a", 1)

    var wg sync.WaitGroup
    // 10 lettori concorrenti.
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            if v, ok := s.Get("a"); !ok || v < 1 {
                fmt.Println("valore inatteso")
            }
        }()
    }
    // 1 scrittore concorrente.
    wg.Add(1)
    go func() {
        defer wg.Done()
        s.Set("b", 2)
    }()
    wg.Wait()

    v, _ := s.Get("b")
    fmt.Println("b =", v)
```

```
}
```

```
b = 2
```

`Get` usa il lock di lettura: dieci lettori possono procedere insieme. `Set` prende il lock di scrittura, che esclude lettori e altri scrittori. Il risultato finale (`b = 2`) è deterministico perché `wg.Wait()` garantisce che lo scrittore abbia terminato.

Confronto — C, Python, Java

Equivalenti: in **C** `pthread_mutex_t` e `pthread_rwlock_t`; in **Java** `synchronized`, `ReentrantLock`, `ReentrantReadWriteLock`; in **Python** `threading.Lock` e `RLock`. La differenza di filosofia: in Go si preferisce, quando possibile, comunicare con channel anziché proteggere memoria condivisa con lock; i mutex restano lo strumento giusto per proteggere uno stato semplice (un contatore, una cache).

8.9 Sincronizzazione con `WaitGroup`

`sync.WaitGroup` permette a un gruppo di goroutine di *attendere collettivamente* un evento prima di proseguire in modo indipendente. È come un **contatore sicuro per la concorrenza** con tre operazioni: `Add`, `Done` e `Wait`.

Due usi tipici, indicati dalle slide:

1. **Pulizia**: assicurarsi che *tutte* le goroutine (`main` compresa) terminino prima della fine del programma;
2. **Pattern BSP** (*Bulk Synchronous Parallel*): le goroutine lavorano in modo indipendente per un po', poi attendono a una *barriera* prima di procedere; lo scambio di dati può avvenire alla barriera.

8.9.1 Uso per la pulizia

Listato 8.19: `06_concurrency/waitgroup.go`

```go
// waitgroup.go - sync.WaitGroup per attendere un gruppo di goroutine
// (slide 64).
package main

import (
    "fmt"
    "sync"
)

func worker(id int, results []int, wg *sync.WaitGroup) {
    defer wg.Done()
    // Ogni goroutine scrive in una posizione DIVERSA della slice: non c'è
    // condivisione dello stesso elemento, quindi non serve un lock.
    results[id] = id * id
}

func main() {
```

```
    results := make([]int, 5)
    var wg sync.WaitGroup

    for i := range results {
        wg.Add(1) // un'unità in più da attendere
        go worker(i, results, &wg)
    }
    wg.Wait() // attende che tutti abbiano chiamato Done

    // Dopo Wait tutte le scritture sono visibili a main.
    fmt.Println(results) // [0 1 4 9 16]
}
```

```
[0 1 4 9 16]
```

Ogni goroutine scrive nella *propria* posizione di `results`: non c'è condivisione dello stesso elemento, quindi non servono lock. Dopo `wg.Wait()` le scritture di tutte le goroutine sono visibili a `main`: `Wait` stabilisce la sincronizzazione necessaria.

8.9.2 Uso come barriera: il pattern BSP

Listato 8.20: `06_concurrency/bsp_barrier.go`

```
// bsp_barrier.go - Bulk Synchronous Parallel con WaitGroup come barriera
// (slide 64).
//
// Pattern BSP: le goroutine lavorano indipendentemente (fase di calcolo),
// poi attendono tutte a una barriera; solo dopo si passa alla fase
// successiva, in cui i dati prodotti sono visibili a tutti.
package main

import (
    "fmt"
    "sync"
)

func main() {
    const n = 4
    partial := make([]int, n)

    // Fase 1: ogni goroutine calcola un risultato parziale.
    var wg sync.WaitGroup
    for i := 0; i < n; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            partial[i] = (i + 1) * 10
        }()
    }
    wg.Wait() // BARRIERA

    // Scambio dati alla barriera: ogni goroutine della fase 2 legge
    // i risultati parziali di tutte le altre.
    total := 0
    for _, p := range partial {
```

```
        total += p
    }
    fmt.Println("totale dopo la fase 1:", total) // 100

    // Fase 2: ogni goroutine usa il totale globale.
    shares := make([]float64, n)
    for i := 0; i < n; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            shares[i] = float64(partial[i]) / float64(total)
        }()
    }
    wg.Wait() // seconda barriera
    fmt.Println("quote:", shares)
}
```

```
totale dopo la fase 1: 100
quote: [0.1 0.2 0.3 0.4]
```

Nella fase 1 ogni goroutine calcola un risultato parziale; la prima `wg.Wait()` è la **barriera**: nessuno passa alla fase 2 prima che tutti abbiano finito. Alla barriera i dati sono scambiati: `main` somma i parziali ($10 + 20 + 30 + 40 = 100$). Nella fase 2 ogni goroutine usa il totale globale per calcolare la propria quota. Il `WaitGroup` è riutilizzabile: il contatore torna a zero e si può ricominciare con nuove chiamate a `Add`.

Confronto — C, Python, Java

`WaitGroup` corrisponde a `pthread_join` su un insieme di thread, a `Thread.join()` (Python e Java) o, più da vicino, al `CountDownLatch` di Java. Per la barriera, `CyclicBarrier` (Java) o `pthread_barrier_t` (C).

8.10 Priorità e scheduling

Le goroutine **non hanno una schedulazione a priorità**: lo scheduler di Go punta su equità e semplicità. Per imporre una priorità occorre realizzarla *a livello applicativo*, con una complessità aggiuntiva. Due approcci per simularla, indicati dalle slide:

1. usare **più channel** per classi di priorità diverse (alta, bassa);
2. usare una **coda con priorità** (*heap*) per ordinare i task per priorità.

L'esempio seguente realizza il primo approccio.

Listato 8.21: `06_concurrency/priority.go`

```
// priority.go - Simulare priorità con più channel (slide 65).
//
// Lo scheduler di Go non ha priorità: la si implementa a livello di
// applicazione. Qui due channel (alta/bassa priorità); il consumatore
// controlla SEMPRE prima il channel ad alta priorità con un select non
// bloccante e passa a quello a bassa priorità solo se il primo è vuoto.
package main

```

```go
import "fmt"

func main() {
    high := make(chan string, 3)
    low := make(chan string, 3)

    // I task a bassa priorità vengono inseriti per primi, di proposito.
    low <- "low-1"
    low <- "low-2"
    high <- "high-1"
    high <- "high-2"

    const total = 4
    for done := 0; done < total; done++ {
        select {
        case t := <-high: // controllato per primo
            fmt.Println("eseguo", t)
        default:
            // Nessun task ad alta priorità: si serve il basso.
            select {
            case t := <-low:
                fmt.Println("eseguo", t)
            default:
            }
        }
    }
}
```

```
eseguo high-1
eseguo high-2
eseguo low-1
eseguo low-2
```

Analisi del codice

I task a bassa priorità sono inseriti per primi, di proposito. Il consumatore, a ogni giro, esegue prima un `select` *non bloccante* sul channel ad alta priorità; solo se questo è vuoto (ramo `default`) tenta il channel a bassa priorità con un secondo `select`. Il risultato è che tutti i task `high` vengono serviti prima di qualunque `low`, indipendentemente dall'ordine di inserimento. Si noti che l'ordine *non è garantito dallo scheduler* ma dalla struttura del codice applicativo, come ci si aspetta.

8.11 Deadlock

Un **deadlock** si verifica quando tutte le goroutine sono bloccate in attesa di eventi che nessuna di loro potrà produrre. L'esempio più semplice: inviare su un channel senza buffer dalla stessa goroutine che dovrebbe riceverne il valore.

Errore comune: errori nella gestione dei channel

Il programma seguente termina *volutamente* con un errore fatale. L'invio `ch <- 1` blocca finché qualcuno riceve, ma l'unica goroutine esistente è proprio quella bloccata: nessuno riceverà mai. Il runtime di Go, quando rileva che *tutte* le goroutine sono bloccate, abortisce il programma.

Listato 8.22: `06_concurrency/pitfall_deadlock.go`

```go
// pitfall_deadlock.go - ERRORE COMUNE: deadlock su channel senza buffer.
//
// ATTENZIONE: questo programma TERMINA VOLUTAMENTE con un errore fatale:
//
//  fatal error: all goroutines are asleep - deadlock!
//
// L'invio su un channel senza buffer blocca finché qualcuno riceve, ma
// l'unica goroutine esistente è quella bloccata: nessuno riceverà mai.
// Il runtime lo rileva e abortisce il programma (exit code 2).
package main

import "fmt"

func main() {
    ch := make(chan int)

    // ERRORE: invio bloccante nella stessa goroutine che dovrebbe ricevere.
    ch <- 1
    fmt.Println(<-ch) // mai raggiunto

    // Correzioni possibili: (1) channel con buffer make(chan int, 1);
    // (2) inviare da un'altra goroutine: go func() { ch <- 1 }().
}
```

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        .../pitfall_deadlock.go:18 +0x36
exit status 2
```

Rimedi: un channel con buffer (`make(chan int, 1)`) oppure inviare da un'altra goroutine (`go func() { ch <- 1 }()`). Si noti che il rilevamento automatico funziona solo quando *tutte* le goroutine sono bloccate: se ne esiste una ancora attiva, una goroutine bloccata resta semplicemente ferma (*goroutine leak*) senza alcun messaggio.

8.12 Riepilogo degli errori comuni nella concorrenza

Errore	Conseguenza e rimedio
`main` termina prima delle goroutine	Le goroutine sono abbandonate. Usare `sync.WaitGroup` o un channel `done`.
Accesso concorrente non protetto	Data race, aggiornamenti persi. Usare `sync.Mutex` o comunicare via channel.
`wg.Add` dentro la goroutine	`Wait` può ritornare troppo presto. Chiamare `Add` prima di `go`.
Invio su channel chiuso	Panic. Chiudere solo dal mittente, una sola volta.
`range` su channel mai chiuso	Blocco permanente. Il mittente deve fare `close`.
Invio/ricezione senza controparte	Deadlock. Usare buffer, un'altra goroutine o `select` con `default`.
Assumere un ordine tra goroutine	Output variabile. L'ordine si ottiene solo con sincronizzazione esplicita.

Capitolo 9

Test e benchmark

9.1 Il package `testing`

Il package `testing` della libreria standard fornisce gli strumenti per scrivere **unit test**. Le convenzioni, interamente imposte dal toolchain:

- i test stanno, di norma, in file il cui nome termina con `_test.go` (per esempio `abs_test.go`); questi file sono compilati solo da `go test` e non finiscono nel programma;
- una funzione di test si chiama `TestXxx`, dove `Xxx` richiama il nome della funzione sotto test, e riceve un parametro `*testing.T`;
- i test si eseguono con `go test`.

Listato 9.1: Il test della slide

```
func TestAbs(t *testing.T) {
    got := Abs(-1)
    if got != 1 {
        t.Errorf("Abs(-1) = %d; want 1", got)
    }
}
```

L'oggetto `t` serve a segnalare i fallimenti: `t.Errorf` marca il test come fallito ma lo lascia proseguire, `t.Fatalf` lo interrompe subito. Non esistono *assert*: si usa un normale `if` e si segnala l'errore.

9.1.1 Esempio pratico

La cartella `07_testing_and_tools` è un package ordinario (`package tools`), non un programma `main`: contiene la funzione sotto test e i file di test.

Listato 9.2: `07_testing_and_tools/abs.go`

```
1  // Package tools contiene la funzione sotto test per gli esempi di
2  // testing e benchmarking (slide 71-72).
3  //
4  // Non è un package main: non si esegue con "go run", ma con
5  //
6  //  go test ./07_testing_and_tools/
7  //  go test -bench=. ./07_testing_and_tools/
8  package tools
9
10 // Abs restituisce il valore assoluto di n.
11 func Abs(n int) int {
12     if n < 0 {
```

```
        return -n
    }
    return n
}

// FillAppend costruisce una slice di n elementi partendo da una slice
// vuota e usando append (può dover riallocare più volte).
func FillAppend(n int) []int {
    var s []int
    for i := 0; i < n; i++ {
        s = append(s, i)
    }
    return s
}

// FillMake costruisce la stessa slice allocando in anticipo la capacità
// con make: una sola allocazione.
func FillMake(n int) []int {
    s := make([]int, 0, n)
    for i := 0; i < n; i++ {
        s = append(s, i)
    }
    return s
}
```

Listato 9.3: 07_testing_and_tools/abs_test.go

```
package tools

import "testing"

// I test stanno in file *_test.go; le funzioni si chiamano TestXxx e
// ricevono un *testing.T.
func TestAbs(t *testing.T) {
    got := Abs(-1)
    if got != 1 {
        // Errorf segna il test come fallito ma lo lascia proseguire.
        t.Errorf("Abs(-1) = %d; want 1", got)
    }
}

func TestAbsPositive(t *testing.T) {
    if got := Abs(5); got != 5 {
        t.Errorf("Abs(5) = %d; want 5", got)
    }
}

// Verifica che le due costruzioni producano lo stesso risultato.
func TestFillEquivalent(t *testing.T) {
    a, b := FillAppend(100), FillMake(100)
    if len(a) != len(b) {
        t.Fatalf("lunghezze diverse: %d vs %d", len(a), len(b))
    }
    for i := range a {
        if a[i] != b[i] {
            t.Fatalf("diverse in posizione %d", i)
        }
```

```
31      }
32  }
```

Si esegue dalla cartella `examples/`:

```
go test -v ./07_testing_and_tools/
```

```
=== RUN   TestAbs
--- PASS: TestAbs (0.00s)
=== RUN   TestAbsPositive
--- PASS: TestAbsPositive (0.00s)
=== RUN   TestFillEquivalent
--- PASS: TestFillEquivalent (0.00s)
PASS
ok      goguide/07_testing_and_tools    (cached)
```

Analisi del codice

`go test` compila il package insieme ai file `_test.go`, esegue ogni funzione `TestXxx` e riporta `PASS` o `FAIL`. Con `-v` si vede ogni test; senza, solo il risultato finale (`ok`). Si noti l'uso di `t.Fatalf` in `TestFillEquivalent`: se le lunghezze differiscono è inutile confrontare gli elementi, quindi si interrompe subito.

Confronto — C, Python, Java

Go incorpora il framework di test nel toolchain: non servono dipendenze esterne come *JUnit* (Java), *pytest*/*unittest* (Python) o *CUnit*/*Unity* (C). Le convenzioni sui nomi sostituiscono le annotazioni e la configurazione.

9.2 Benchmark

Go fornisce anche strumenti per misurare le **prestazioni** del codice con i *benchmark*. I benchmark stanno nei file `_test.go` e il loro nome inizia con `Benchmark`; ricevono un `*testing.B`.

Listato 9.4: Il benchmark della slide

```
func BenchmarkXxx(b *testing.B) {
    for i := 0; i < b.N; i++ {
        Xxx(...)
    }
}
```

Il benchmark esegue la funzione `b.N` volte per raccogliere dati sulle prestazioni; il runner *regola* `b.N`, ripetendo l'esecuzione più volte finché la misura è accurata. Si lanciano con `go test -bench=.`.

9.2.1 Esempio: `make` contro `append`

Le slide propongono di confrontare la costruzione di una slice con `append` partendo da una slice vuota (che può dover riallocare più volte) con la preallocazione della capacità tramite `make`. Le funzioni misurate sono `FillAppend` e `FillMake` nel file `abs.go` qui sopra.

Listato 9.5: 07_testing_and_tools/benchmark_test.go

```go
package tools

import "testing"

// sink è una variabile a livello di package: assegnarvi il risultato
// impedisce al compilatore di scartare o allocare sullo stack la slice
// (altrimenti, con inlining e dimensione costante, il benchmark di make
// risulterebbe senza allocazioni e il confronto sarebbe falsato).
var sink []int

// Benchmark: il runner sceglie b.N in modo adattivo, ripetendo il corpo
// finché la misura è stabile.
//
//  go test -bench=. ./07_testing_and_tools/

func BenchmarkFillAppend(b *testing.B) {
    for i := 0; i < b.N; i++ {
        sink = FillAppend(1000)
    }
}

func BenchmarkFillMake(b *testing.B) {
    for i := 0; i < b.N; i++ {
        sink = FillMake(1000)
    }
}
```

Esecuzione (macchina usata per la verifica: CPU Intel Core i7-13700H, Windows):

```
go test -run NONE -bench "." -benchmem ./07_testing_and_tools/
```

```
BenchmarkFillAppend-20   220239   5957 ns/op   25208 B/op   12 allocs/op
BenchmarkFillMake-20     513416   3160 ns/op    8192 B/op    1 allocs/op
```

(Le colonne, separate da tabulazioni nell'output originale, sono qui allineate con spazi; i valori sono quelli misurati: iterazioni, tempo per operazione, byte e numero di allocazioni per operazione.)

Analisi del risultato

- I tempi assoluti (`ns/op`) **dipendono dalla macchina** e variano da esecuzione a esecuzione: i numeri sopra sono una misura reale ma non riproducibile identica altrove. Ciò che si ripete è il *confronto*: la versione con `make` è più veloce.
- La differenza si spiega con le allocazioni: `append` su una slice vuota riallocava 12 volte (cresce a salti, copiando ogni volta l'array), occupando 25 208 byte allocati in totale; `make` con capacità nota alloca una sola volta, 8 192 byte, esattamente lo spazio di 1000 `int` da 8 byte.
- **Perché `sink`?** Nella prima versione del benchmark il risultato veniva scartato: il compilatore, che inline-a funzioni piccole e conosce la dimensione costante, poteva allocare la slice sullo stack e il benchmark di `make` riportava *zero* allocazioni, falsando il confronto. Assegnando il risultato a una variabile di package (`sink`) la slice «scappa» sullo heap e

la misura è equa. È una lezione generale: un benchmark misura anche le ottimizzazioni del compilatore, quindi va scritto in modo da impedire che scartino il lavoro da misurare.

Errori comuni nei test e nei benchmark

- Il file non termina con `_test.go`, oppure la funzione non inizia con `Test`/`Benchmark` (con la maiuscola): `go test` la ignora silenziosamente e si crede di avere test che non esistono.
- Dimenticare di usare `b.N` come numero di iterazioni nel ciclo: il benchmark misurerebbe una sola esecuzione.
- Confrontare tempi misurati su macchine diverse o con altro carico in esecuzione.

9.3 Argomenti citati ma non sviluppati

L'ultima parte delle slide elenca argomenti *ulteriori*, la cui trattazione completa è rimandata ad altro materiale: **RPC in Go** (le slide rimandano alle slide del corso sulla comunicazione e sulle RPC) e, come esempio dei «molti altri aspetti da esplorare», il **supporto HTTP del package `net/http`**. Questa guida li menziona senza svilupparli, per restare nel perimetro delle slide; di `net/http` compare solo il tipo `http.ResponseWriter` / `*http.Request` nell'esempio del middleware (Sezione 4.5.2). Moduli, funzioni variadiche, gestione degli errori, strumenti e test, anch'essi elencati come «argomenti aggiuntivi», sono invece trattati nei capitoli 2, 4, 7 e in questo.

Conclusione e riepilogo

Go è un linguaggio piccolo, con poche regole applicate in modo rigido. Se si guardano insieme i capitoli, le scelte di progetto si ricompongono in un disegno coerente: *semplicità* (un solo ciclo, nessuna classe, nessuna aritmetica sui puntatori), *sicurezza* (tipizzazione statica, garbage collector, controllo dei limiti, valori zero) e *scalabilità* (goroutine e channel come cittadini di prima classe).

Riepilogo dei concetti principali

Concetto	**Punto chiave da ricordare**
Package e visibilità	Maiuscola iniziale ⇒ esportato. Esecuzione da `main.main`.
Toolchain	`go run`, `go build`, `go install`; i moduli con `go.mod`; formattazione con `gofmt`.
Variabili	`var` ovunque, `:=` solo nelle funzioni; valore zero per ogni tipo.
Tipi	Conversioni sempre esplicite; inferenza dal lato destro.
Funzioni	Risultati multipli, risultati con nome, variadiche, anonime, closure che catturano variabili.
Controllo di flusso	Un solo `for`; `if` con istruzione iniziale; `switch` senza fallthrough; `defer` in ordine LIFO con argomenti valutati subito.
Puntatori	`&` e `*`, nessuna aritmetica, `nil`, sicuro restituire l'indirizzo di una locale.
Struct, array	Valori, copiati nelle assegnazioni; la lunghezza dell'array fa parte del tipo.
Slice	Vista *(puntatore, len, cap)* su un array; `append` va *riassegnato*.
Map	Riferimento a tabella hash; chiave assente ⇒ valore zero; usare `v, ok`; ordine non specificato.
Metodi e interfacce	Receiver per valore o puntatore; interfacce *implicite* basate sul comportamento.
Errori	Valori di tipo `error`, ultimo risultato; `if err != nil`.
Goroutine	Thread leggeri gestiti dal runtime; `main` non le attende; nessun ordine garantito.
Channel	Code tipizzate e thread-safe; senza buffer sincronizzano; `close` solo dal mittente; `range` fino alla chiusura.
`select`	Attesa su più channel; scelta casuale tra i pronti; `default` per non bloccare.
Timer	Un canale `C` che scatta dopo una durata; `Stop()` per annullare.
Sincronizzazione	`sync.Mutex`, `sync.RWMutex`, `sync.WaitGroup` (anche come barriera BSP).
Test e benchmark	File `_test.go`, `TestXxx(*testing.T)`, `BenchmarkXxx(*testing.B)`; `go test`.

Per proseguire

Le slide chiudono ricordando che questa è *solo un'introduzione*: restano da esplorare, tra l'altro, le RPC in Go e il supporto HTTP del package `net/http`. Il modo più efficace per consolidare quanto appreso è modificare gli esempi della cartella `examples/`: cambiare le capacità delle slice e osservare `append`, togliere un `close` e provocare un deadlock, rimuovere il mutex dal contatore e osservare la data race.

Appendice A

Mappa tra slide, guida ed esempi

La tabella associa i numeri delle slide (quelli stampati sulle slide) ai capitoli della guida e ai file in `examples/`. Le slide 1–2 contengono solo cenni storici e classifiche (omessi) e la slide 73 solo riferimenti bibliografici e web (omessa).

Slide	Argomento	Cap.	Esempi
1–5	Obiettivi, eredità da C, novità, sistemi distribuiti	1	—
6	Go per applicazioni cloud-native (Go Cloud)	1	frammento non eseguibile
7	Editor, plugin, `gopls`	1	—
8–9	Hello world, package, `main`, `fmt`	2	`helloworld.go`
10	Toolchain: `go run/build/install`	2	—
11	Package e visibilità	2	`imports.go`
12	Import	2	`imports.go`
13–15	Funzioni, risultati multipli, errori	4	`functions.go`, `swap.go`, `result_and_error.go`
16	Valori di ritorno con nome	4	`named_returns.go`
17	Variabili	3	`variables.go`
18	Tipi, conversioni, inferenza	3	`types.go`
19–22	Controllo di flusso, `for`, `echo`	5	`for_loops.go`, `echo.go`
23–24	`if/else`	5	`if_else.go`
25	`switch`	5	`switch.go`
26	`defer`	5	`defer.go`, `defer_lifo.go`
27	Puntatori	6	`pointers.go`
28	Struct e array	6	`structs.go`, `arrays.go`
29–31	Slice, `len/cap`, `make`	6	`slices.go`, `making_slice.go`
32–33	`append`	6	`append.go`, `slice_pitfalls.go`
34	Map	6	`maps.go`
35–36	`range`	6	`range.go`, `range2.go`
37–39	Funzioni anonime, closure	4	`anonymous.go`, `closure.go`, `closure_fib.go`
40	Closure come middleware	4	`middleware.go`
41	Metodi	7	`methods.go`
42–45	Interfacce	7	`interface.go`, `interface_slice.go`
46–48	Concorrenza, goroutine, scheduler	8	`goroutine_termination.go`, `goroutine_wait.go`, `gomaxprocs.go`
49–52	Channel	8	`channel_sync.go`, `channel_sum.go`, `channel_fib.go`
53	Channel con/senza buffer	8	`channel_buffered.go`
54–55	`close`, `range` su channel	8	`channel_close.go`, `channel_range.go`, `pitfall_send_closed.go`
56–59	`select`, `default`	8	`channel_fib_select.go`, `select_multiplex.go`, `select_default.go`

Slide	Argomento	Cap.	Esempi
60	Timer	8	`timers.go`
61–63	Mutex con channel, `sync.Mutex`, `RWMutex`	8	`mutex_using_channel.go`, `mutex.go`, `race_counter.go`, `rwmutex.go`
64	`WaitGroup`, BSP	8	`waitgroup.go`, `bsp_barrier.go`
65	Priorità	8	`priority.go`
66	Argomenti aggiuntivi (RPC, HTTP: citati)	9	—
67	Moduli	2	`go.mod`
68	Funzioni variadiche	4	`variadic.go`
69	Gestione degli errori	7	`errors.go`
70	Regole del compilatore, `gofmt`, `goimports`, `godoc`	2	—
71–72	Test e benchmark	9	`abs.go`, `abs_test.go`, `benchmark_test.go`

Esempi aggiuntivi non presenti nelle slide, scritti per chiarire concetti già trattati: `interface_slice.go`, `channel_sync.go`, `select_multiplex.go`, `bsp_barrier.go`, `priority.go`, `race_counter.go`, `slice_pitfalls.go`, `gomaxprocs.go`, `defer_lifo.go`, `anonymous.go`, `pitfall_deadlock.go`, `pitfall_send_closed.go`, oltre agli esempi sugli errori comuni inseriti negli altri file.

Appendice B

Note della verifica incrociata

B.1 Metodo

La verifica ha confrontato quattro elementi: le slide originali, il testo di questa guida, i file `.go` e gli output. In concreto:

- tutte le 73 slide sono state lette prima di scrivere, e ogni argomento tecnico è stato associato a una sezione (Appendice A);
- i listati della guida sono inclusi *dai file reali* (`examples/`), quindi la corrispondenza è esatta per costruzione;
- gli output mostrati sono stati ottenuti eseguendo i programmi con Go 1.27.3; quelli non deterministici sono stati eseguiti più volte e indicati come tali;
- ogni file è stato formattato con `gofmt` e controllato con `go vet`.

B.2 Errori e imprecisioni trovati nelle slide

Per ciascun punto si distingue il **problema di trascrizione** (errore materiale nella slide) dalla **correzione o precisazione tecnica** introdotta dalla guida.

#	Slide	Osservazione	Trattamento nella guida
1	58	*Trascrizione.* `fibonacci(c, quit)` usa la variabile `c`, mai dichiarata (si dichiara `ch`); non compila (`undefined: c`).	Corretto in `fibonacci(ch, quit)` in `channel_fib_select.go`; segnalato nel Capitolo 8.
2	56	*Trascrizione.* `case mgs1 := <-ch1` è un refuso per `msg1`.	Usato `msg1`; segnalato nel Capitolo 8.
3	48	*Precisazione tecnica.* «Set `runtime.GOMAXPROCS` environment variable» mescola due cose: la variabile d'ambiente si chiama `GOMAXPROCS`; `runtime.GOMAXPROCS(n)` è la funzione.	Distinte le due forme (Sez. 8.2); nessuna modifica al concetto.
4	11	*Aggiornamento.* Il messaggio d'errore per `math.pi` oggi è `undefined: math.pi (but have Pi)`; la slide riporta la forma più breve.	Riportato il messaggio ottenuto con Go 1.27.3.
5	36	*Precisazione.* L'output `a -> apple`, `b -> banana` non è garantito: l'ordine di iterazione di una map è non specificato.	L'esempio itera su un elenco di chiavi ordinato a mano; spiegato nel Capitolo 6.

#	Slide	Osservazione	Trattamento nella guida
6	33	*Precisazione.* «Di norma il doppio della capacità precedente»: nell'esecuzione reale `append(s, 2, 3, 4)` su una slice di capacità 2 dà capacità **6**, non 5 né 4.	Discusso in `append.go` e nel Capitolo 6: la regola è un dettaglio d'implementazione.
7	50, 53	*Precisazione.* «Invio e ricezione bloccano finché l'altro lato è pronto» vale solo per channel senza buffer.	Chiarito con un box nella Sezione 8.4.
8	15	*Dipendenza dall'ambiente.* `os.ReadFile("example.txt")` presuppone un file esistente.	Esempio reso autonomo creando il file in una directory temporanea.
9	40	*Frammento incompleto.* `timed` è mostrato senza `hello`, imports e server.	Reso eseguibile con `httptest`, senza rete.
10	62	*Segnalazione di `go vet`.* Il codice compila e funziona, ma vet segnala `return copies lock value` su `NewLock`.	Codice mantenuto come nella slide; il warning è spiegato (Sez. 8.8).
11	30	*Precisazione.* «Compile or run-time error if array length is exceeded»: l'errore è a compile time solo per indici *costanti* su array; altrimenti è un panic a runtime.	Entrambi i casi mostrati con i messaggi reali (Cap. 6).
12	48	*Dato non verificabile qui.* Le stime «circa 5× più veloci, 500× meno memoria» non sono state misurate.	Riportate come *dato delle slide*, non verificato.

B.3 Contenuto delle slide deliberatamente omesso

Come richiesto, non compaiono: la storia del linguaggio (anno di concezione e autori, slide 1), le classifiche di popolarità (slide 2), l'elenco dei riferimenti bibliografici e web (slide 73) e i collegamenti a siti esterni. Il frammento Go Cloud (slide 6) è riportato come testo ma non ha un file eseguibile perché dipende da una libreria esterna e da servizi cloud.

B.4 Verifica degli esempi

- Tutti i programmi della cartella `examples/` compilano ed eseguono con Go 1.27.3 (Windows, amd64).
- Due programmi terminano *volutamente* con un errore, e sono marcati nel codice e nel testo: `pitfall_deadlock.go` (deadlock, exit code 2) e `pitfall_send_closed.go` (panic, exit code 2).
- Gli esempi con output non deterministico sono: `goroutine_termination.go`, `goroutine_wait.go`, `select_multiplex.go`, `race_counter.go`, l'intercalarsi delle righe in `channel_close.go` e i tempi di `middleware.go` e dei benchmark.
- Non sono stati eseguiti con il *race detector* (`-race`), che richiede il supporto di CGO non disponibile nell'ambiente di verifica. La data race di `race_counter.go` è dimostrata dal risultato errato, non dallo strumento.